



ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ
UNIVERSITY OF PIRAEUS

Project Q-Catch

Christos Karatsalos
mtn2510

Konstantinos Mpouros
mtn2517

Supervising Professor, George Vouros
Course, Intelligent Agents and Multi-Agent Systems

MSc in Artificial Intelligence
University of Piraeus & NCSR "DEMOKRITOS", Greece

September 28, 2026

Contents

Abstract	vi
1 Introduction	1
2 Theoretical & Mathematical Background	2
2.1 Markov Decision Processes and Markov Games	2
2.1.1 Markov Decision Processes (MDPs)	2
2.1.2 Markov Games (Stochastic Games)	3
2.1.3 Multi-Agent Learning Settings: Cooperative vs. Competitive	3
2.2 Single-Agent Q-Learning (Tabular)	4
2.3 Policies and Exploration	5
2.4 Reward Structures and Shaping	6
2.5 Minimax-Q Learning	7
2.6 Nash-Q Learning	8
2.7 Dyna-Q: Integrating Learning and Planning	9
2.7.1 Dyna-Q+ (Dynamic Environments and Exploration Bonus)	10
2.8 Deep Q-Networks (DQN)	11
3 Problem Setup and Evaluation Protocol	12
3.1 Gridworld Environment Overview	12
3.2 Reward Structure	13
3.3 Training Protocol	14
3.3.1 Periodic Evaluation Protocol	16
3.4 Logging and Experimental Settings	16
3.4.1 Data Logging Scheme	17
3.4.2 Experimental Parameters and Reproducibility	17
4 Base Chasing	19
4.1 Scenario Overview	19
4.2 Environment Specification (TagEnv)	19
4.2.1 Grid, Coordinates, and State Space	19
4.2.2 Action Space and Simultaneous Dynamics	19

4.2.3	Stochastic Action Failure	20
4.2.4	Capture Conditions	20
4.2.5	Episode Termination	20
4.3	Reward Design and Shaping	20
4.3.1	Decomposed Reward Components	20
4.3.2	Step-Level Diagnostics (info Structure)	21
4.4	Tabular State Encoding	21
4.5	Agents in the Base Chasing Scenario	22
4.5.1	Fictitious Play Agent (FP)	22
4.5.2	Minimax-Q Agent	22
4.6	Training, Evaluation, and Outputs (Base Chasing)	23
4.6.1	Matchups and Execution	23
4.6.2	Periodic Evaluation Protocol	23
4.6.3	Step-Level Logging (Excel Output)	24
5	Wumpus Chase	25
5.1	Scenario Overview and Design Goals	25
5.2	Map Layout System and World Elements	25
5.3	Extended Environment Specification	26
5.3.1	Grid, Coordinates, and State	26
5.3.2	Action Space and Stochastic Dynamics	26
5.3.3	Reset and Valid Start States	26
5.3.4	Terminal Outcome Priority (No-Draw Runtime)	27
5.4	Perception Model	27
5.5	Reward Design and Decomposition	27
5.5.1	Reward Parameters and Interface Convention	27
5.5.2	Outcome, Hazard, and Shaping Terms	28
5.5.3	Step-Level Diagnostics (info Structure)	28
5.6	Tabular State Encoding and Game-Theoretic Utilities	29
5.6.1	Tabular State Encoding	29
5.6.2	Maximin Linear Program (LP) for Security Policies	29
5.7	Learning Agents Supported in Wumpus Chase	29
5.7.1	Fictitious Play (FP)	29
5.7.2	Minimax-Q	29
5.7.3	Nash-Q	30
5.7.4	Dyna-Q and Dyna-Q+	30
5.7.5	Deep Q-Networks (Optional DQN Stack)	30
5.8	Training, Evaluation, and Logging Protocol	30
5.8.1	Experimental Matchups (What We Trained)	30
5.8.2	Training Loop and Update Semantics	31
5.8.3	Evaluation Protocol and Metrics	31
5.8.4	Step-Level Logging for Post-Hoc Analysis	32

5.9 Extension: Opponent Diversity and Robustness	32
5.9.1 Research Question and Experimental Scope	32
5.9.2 State Representation, Rewards, and Common Learner	33
5.9.3 Opponent Banks and Training Conditions	33
5.9.4 Evaluation Protocol and Experimental Budget	34
5.9.5 Limitations and Further Experiments	35
6 Experiments Results	36
6.1 Evaluation Results: Chase	36
6.1.1 Rolling performance across evaluation episodes	36
6.1.2 Steps-to-capture distributions	36
6.1.3 Controlling for initial difficulty: opening Manhattan distance	37
6.1.4 Empirical move-failure rate under p_{fail}	38
6.1.5 Qualitative spatial behavior: visitation and capture locations	39
6.1.6 Additional evaluation diagnostics in the notebooks (not included here)	40
6.2 Evaluation Results: Wumpus Chase	40
6.2.1 Overall performance by role	40
6.2.2 Terminal outcomes: what ends the episode	42
6.2.3 Behavioral diagnostics	42
6.2.4 Summary of findings	44
6.3 Evaluation Results: Opponent Diversity and Robustness	44
6.3.1 Pilot Results and Role-Dependent Interpretation	45
7 Conclusion	49
Bibliography	50

List of Figures

6.1	Chase evaluation (rolling metrics) under $p_{\text{fail}} = 0.1$: rolling capture rate (left) and rolling steps (right) for all run configurations.	37
6.2	Chase evaluation (rolling metrics) under $p_{\text{fail}} = 0.2$: rolling capture rate (left) and rolling steps (right) for all run configurations.	37
6.3	Histogram of steps-to-capture (captured episodes only), faceted by run configuration (columns) and failure probability p_{fail} (rows).	38
6.4	Rolling opening Manhattan distance (step 1) across evaluation episodes, stratified by run and p_{fail}	38
6.5	Rolling move-failure rate versus evaluation episode id for catcher (c_fail) and runner (r_fail), faceted by run (columns) and failure probability p_{fail} (rows).	39
6.6	Spatial diagnostics for minimaxq_vs_minimaxq at $p_{\text{fail}} = 0.1$: catcher visitation (left), runner visitation (center), and capture locations (right).	40
6.7	Evaluation aggregate: win rate by agent and role (Chaser vs Runner).	41
6.8	Evaluation aggregate: average terminal reward by agent and role.	41
6.9	Evaluation aggregate: average terminal steps by agent and role.	42
6.10	Evaluation aggregate: terminal reason mix by agent-role (stacked rates).	43
6.11	Terminal step distribution by terminal reason. Timeouts concentrate near the horizon, while capture/treasure endings occur substantially earlier.	44
6.12	Hazard-type mix among hazard endings (pit vs Wumpus), split by side (Chaser vs Runner).	45
6.13	Evaluation action mix by role and run (stacked action shares).	46
6.14	Control-zone heatmaps for dynaq_plus_selfplay , $p_{\text{fail}} = 0.1$: Runner visitation density under pressure ($d_{AB} \leq 2$) vs outside pressure ($d_{AB} > 2$).	47
6.15	Evaluation aggregate: favorable distance progress rate by agent and role.	47
6.16	Evaluation aggregate: mean goal distance by agent and role.	48
6.17	Train vs eval gap: Runner opening-rate difference (eval–train) by run id.	48

List of Tables

- 3.1 Common Environment and Training Parameters 18
- 5.1 Training conditions for the opponent-diversity extension. Conditions C and D sample noise levels uniformly; E adapts their joint distribution with opponent families. 34
- 6.1 Mean primary-endpoint win rates over three training seeds. Bold values identify the largest observed mean within each role and do not establish superiority over every other condition. 45

Abstract

This project studies multi-agent reinforcement learning (MARL) in adversarial gridworld pursuit–evasion settings, using Q-learning-based methods across two environments. The first environment is a base Chase scenario where a Chaser (Agent A) attempts to capture a Runner (Agent B) under stochastic dynamics and obstacles. The second environment extends the chase game into a Wumpus-style gridworld that introduces lethal hazards (pits and a Wumpus), perceptual cues, and a secondary terminal objective (treasure), with strict terminal-outcome priority to avoid draw ambiguity. We implement and compare a suite of tabular MARL agents, including Fictitious Play (FP), Minimax-Q, Nash-Q, and planning-augmented variants (Dyna-Q and Dyna-Q+), alongside an optional deep-RL baseline (DQN). A unified training and periodic evaluation protocol is used, with step-level logging to enable post-hoc analysis of performance and behavior. In the base Chase scenario, evaluation shows stable high-capture regimes for several matchups and highlights that strategic interaction can produce timeout-dominated outcomes in specific pairings. In the Wumpus Chase scenario, results are strongly role-dependent: planning-based methods are most effective for the Chaser, while Dyna-Q+ achieves the strongest Runner performance by sustaining long-horizon survival and converting it into favorable terminal outcomes. Overall, the experiments demonstrate how environment structure (hazards, terminal logic) and agent role shape the effectiveness of MARL methods, and how diagnostic logging (terminal reasons, action tendencies, and distance-based measures) supports mechanistic interpretation beyond win rates.

Chapter 1

Introduction

Multi-agent reinforcement learning (MARL) extends single-agent reinforcement learning to settings where multiple decision-making agents interact. In these environments, the learning problem becomes inherently non-stationary from an individual agent’s perspective, since the environment dynamics depend on the evolving policies of other agents. This project explores MARL in an adversarial chasing context, implementing and evaluating Q-learning-based methods across two scenarios. The first is a base chasing scenario: a two-player predator–prey grid world in which one agent (“Agent A”) attempts to capture the other (“Agent B”). The second is an extended Wumpus Chase scenario inspired by the classic Wumpus World, where the chase game is enriched with environmental hazards and perceptual cues. In both cases, the emphasis is on model-free learning algorithms from the Q-learning family, including multi-agent extensions.

The implementation consists of a custom grid-world environment and a suite of learning agents. In the base scenario, agents move on a grid with obstacles, and Agent A learns to pursue and capture Agent B. The extended Wumpus Chase scenario increases the complexity by introducing Wumpus-style elements such as deadly pits and a hazardous Wumpus creature, requiring agents to balance adversarial pursuit with environmental awareness and risk. We implemented standard tabular Q-learning baselines and multi-agent variants, including Minimax-Q for zero-sum interactions and Nash-Q for general-sum settings. To improve learning efficiency through planning, we incorporated model-based enhancements via Dyna-Q and its exploration-augmented variant Dyna-Q Plus (Dyna-Q+). Finally, we also experimented with a lightweight Deep Q-Network (DQN) agent as an initial step toward function approximation beyond the tabular setting. A complete training and evaluation pipeline was developed to compare performance across methods in self-play and competitive matches.

Chapter 2

Theoretical & Mathematical Background

2.1 Markov Decision Processes and Markov Games

2.1.1 Markov Decision Processes (MDPs)

A Markov Decision Process (MDP) is a mathematical model for sequential decision-making in stochastic environments with the Markov property, meaning that future states depend only on the current state and action, and not on the past history [1]. Formally, an MDP is defined by the tuple (S, A, P, R, γ) , where S is a finite set of states, A is a finite set of actions available to the decision-maker (agent), $P(s' | s, a)$ denotes the transition probability of moving from state s to state s' when action a is taken, $R(s, a)$ is the immediate reward received after taking action a in state s , and $\gamma \in [0, 1)$ is the discount factor that specifies the present value of future rewards [2].

The objective in an MDP is to find a policy $\pi : S \rightarrow A$ (or $\pi : S \rightarrow \Delta(A)$ in the stochastic case, where $\Delta(A)$ denotes a probability distribution over actions) that maximizes the expected cumulative discounted reward, also known as the return. An optimal policy π^* yields the optimal state-value function

$$V^*(s) = \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right]$$

for each state $s \in S$ [1].

The theory of dynamic programming provides Bellman's optimality equations for MDPs. In particular, the optimal state-value function V^* satisfies

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} P(s' | s, a) [R(s, a) + \gamma V^*(s')]$$

for all $s \in S$ [1]. Similarly, the optimal action-value function $Q^*(s, a)$, which represents the expected return obtained by taking action a in state s and thereafter following the optimal policy, satisfies the Bellman optimality condition:

$$Q^*(s, a) = \sum_{s' \in S} P(s' | s, a) \left[R(s, a) + \gamma \max_{a' \in A} Q^*(s', a') \right], \quad (2.1)$$

for all $s \in S$ and $a \in A$. These equations underpin many reinforcement learning algorithms by providing targets for iterative value updates [2]. Solving an MDP therefore amounts to computing the optimal action-value function Q^* (or equivalently the optimal state-value function V^*), from which an optimal policy can be derived, for example by selecting in each state an action that maximizes $Q^*(s, a)$.

2.1.2 Markov Games (Stochastic Games)

Markov games, also known as *stochastic games*, extend the MDP framework to multi-agent environments [3]. A Markov game for N agents is defined by the tuple

$$(S, A^1, \dots, A^N, P, R^1, \dots, R^N, \gamma)$$

, where S is the state set as before, A^i is the action set available to agent i (each agent may have different actions available), $P(s' \mid s, a^1, \dots, a^N)$ is the state transition probability when the joint-action $(a^1, \dots, a^N)$ is taken by the N agents in state s , and $R^i(s, a^1, \dots, a^N)$ is the reward received by agent i from that transition [4]. The discount factor γ is usually shared for all agents (assuming a common horizon). Markov games were first formalized by Shapley [3], who proved foundational results for the two-player zero-sum case, and they provide a general framework for multi-agent reinforcement learning and dynamic game modeling.

In a Markov game, each agent i aims to maximize its own expected discounted sum of rewards, which may be in conflict or in cooperation with other agents' goals. A key concept for multi-agent settings is the notion of an equilibrium. For example, in a two-player zero-sum Markov game, an optimal equilibrium value function can be defined via a minimax condition: there exists a value function $V(s)$ such that $V(s) = \max_{a^1 \in A^1} \min_{a^2 \in A^2} Q(s, a^1, a^2)$, where $Q(s, a^1, a^2)$ is the payoff (e.g., reward or expected return) for the row player (agent 1) when actions a^1, a^2 are taken in state s [4]. In general-sum (non-zero-sum) games with more than one agent, a common solution concept is the Nash equilibrium: a joint policy $(\pi^1, \dots, \pi^N)$ (one policy per agent) is a Nash equilibrium if no agent can unilaterally deviate to improve its own expected return [5]. Existence of Nash equilibria in stochastic games is guaranteed under certain conditions (e.g., for discounted rewards) [?], but the equilibria may not be unique. Finding or learning equilibria in Markov games is substantially more complex than solving single-agent MDPs, due to the coupled interactions between agents' decisions.

2.1.3 Multi-Agent Learning Settings: Cooperative vs. Competitive

Multi-agent reinforcement learning (MARL) scenarios are often categorized by the reward structure and degree of cooperation or competition between agents. Fully cooperative settings are those in which all agents share the same reward function (i.e., $R^1 = R^2 = \dots = R^N$ for all transitions), so agents have a common goal. In this case, a multi-agent problem can be viewed as a team of agents collectively forming a single joint policy to maximize a shared return [6]. A classic approach to fully cooperative MARL is to treat it as a decentralized MDP: agents may still need to learn to coordinate their actions, but there is no fundamental game-theoretic conflict. However, even in cooperative settings, learning can be non-trivial due to the need for coordination (resolving which agent takes which role, etc.). Studies have shown that naive independent learning (each agent learning as if others are part of the environment) can converge to suboptimal outcomes in cooperative tasks that require coordinated action, if agents do not properly account for each other [6].

By contrast, competitive or adversarial settings introduce conflicting goals. The extreme case is a two-player zero-sum game, where one agent’s gain is the other’s loss (e.g., $R^2 = -R^1$ for all outcomes). Many competitive scenarios can be modeled as zero-sum Markov games. In such games, the appropriate solution concept is the minimax value mentioned above, and an optimal policy for one agent is typically a best response to an adversarial opponent that tries to minimize that agent’s payoff. The existence of a well-defined value function $V(s)$ for zero-sum discounted Markov games is analogous to the value in classical matrix games or repeated games, and can be computed (in principle) via algorithms that extend dynamic programming to the minimax case [4]. Competitive MARL problems are challenging because an agent must learn to anticipate and respond to an adversary’s actions, often leading to stable strategies that may involve randomization (mixed policies) to avoid exploitation.

More generally, general-sum (or partially cooperative, partially competitive) settings allow each agent to have an independent reward function with neither full alignment nor complete opposition. These are the most general stochastic games, where agents might form temporary alliances or have incentives to cooperate up to a point. In general-sum games, multiple Nash equilibria may exist and outcomes can be inefficient (analogous to social dilemmas in game theory). Learning in general-sum settings often seeks equilibrium policies (e.g., Nash equilibria) or some targeted solution (such as Pareto optimal points, correlated equilibria, etc., depending on the context). The presence of multiple possible equilibria means that learning dynamics can be highly non-stationary and history-dependent, as agents might oscillate between strategies unless an equilibrium selection or coordination mechanism is in place [5].

A fundamental difficulty in MARL is that the environment, from any single agent’s perspective, is non-stationary when other learning agents are present [6]. This violates the usual MDP assumption of a stationary transition/reward distribution, complicating convergence guarantees. Consequently, specialized learning algorithms and theoretical results are needed for multi-agent settings, as discussed in later sections (e.g., Minimax-Q, Nash-Q).

2.2 Single-Agent Q-Learning (Tabular)

Q-learning is a foundational model-free reinforcement learning algorithm for single-agent MDPs. It was introduced by Watkins [7] as a way for an agent to learn optimal action values (Q-values) through incremental updates, without requiring a model of the environment’s dynamics. In tabular Q-learning, the agent maintains a table of $Q(s, a)$ values for each state-action pair. Initially, these values can be arbitrary (often zero or small random numbers). The agent then interacts with the environment in discrete time steps, and updates its Q-values using the observed rewards and transitions.

The core update rule for Q-learning is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r(s, a) + \gamma \max_{a'} Q(s', a'); -; Q(s, a) \right], \quad (2.2)$$

where s is the current state, a is the action taken, s' is the next state encountered, and $r(s, a)$ is the reward received for taking action a in s [7]. Here $\alpha \in (0, 1]$ is a learning rate (step-size parameter). The term in brackets is the temporal-difference (TD) error, which measures the difference between the current Q-value estimate and a one-step bootstrap

estimate $r + \gamma \max_{a'} Q(s', a')$. The update rule (2.2) moves the current estimate $Q(s, a)$ towards this target.

Over repeated updates, Q-learning performs a form of stochastic approximation to the true optimal Q^* function. A key property of Q-learning is that it is an off-policy algorithm: the update uses the maximum Q value at the next state ($\max_{a'} Q(s', a')$), which corresponds to assuming the agent will follow the optimal policy henceforth, regardless of what action was actually taken in s' . This allows Q-learning to learn the optimal value function independent of the policy being used to explore the environment [2].

Under certain standard conditions, Q-learning can be proven to converge to Q^* with probability 1 as the number of visits to each state-action pair approaches infinity and the learning rate α is decayed appropriately (e.g., satisfying the Robbins-Monro conditions) [7]. In particular, one sufficient condition is that every state-action pair is visited infinitely often in the limit (this typically requires some form of persistent exploration), and $\sum_t \alpha_t(s, a) = \infty$, $\sum_t [\alpha_t(s, a)]^2 < \infty$ for each (s, a) . Watkins and Dayan [7] provide a convergence proof assuming a decaying exploration probability. Subsequent analyses tightened these results and provided finite-time convergence rate bounds in certain settings, but the classical result is that tabular Q-learning is guaranteed to find the optimal policy in the limit, given enough exploration and a properly decreasing learning rate.

2.3 Policies and Exploration

A policy π in reinforcement learning is a strategy used by an agent to decide which action to take in each state. Formally, in the stochastic case, a policy is a mapping from states to a probability distribution over actions, $\pi(a|s) = \Pr a \text{ is chosen in state } s$. In the deterministic case, a policy is a mapping $s \mapsto a$. The goal of RL algorithms is often to learn an optimal policy π^* that maximizes expected return. However, during learning the agent faces the fundamental exploration-exploitation trade-off [8]: it must choose between exploiting the current knowledge (taking actions that currently seem best) and exploring new actions that might yield higher rewards in the long run.

To ensure sufficient exploration of the state-action space, a common approach is to use an ϵ -greedy policy [2]. Under ϵ -greedy exploration, the agent predominantly chooses the current best-known action but occasionally explores a random alternative. Specifically, at state s , with probability $1 - \epsilon$ the agent selects $a = \arg \max_{a'} Q(s, a')$ (the greedy action with respect to its current Q -values), and with probability ϵ it selects a uniformly random action $a \in A$. The parameter $\epsilon \in [0, 1]$ controls the exploration rate; ϵ may be fixed or annealed (decayed) over time. In practice, one often starts with a relatively high ϵ (e.g., 0.1 or 0.2 meaning 10-20% random actions) to encourage broad exploration, and then gradually reduces ϵ as learning progresses to focus more on exploitation of the learned policy. This annealing yields a policy that is *GLIE* (greedy in the limit with infinite exploration) – a condition which helps guarantee convergence in algorithms like Q-learning by ensuring that eventually the policy converges to greedy (exploitation) but all actions continue to be sampled infinitely often in the limit [7].

Besides ϵ -greedy, other exploration strategies exist. One popular alternative is the softmax (Boltzmann) exploration, where the agent chooses action a with probability proportional to $\exp(Q(s, a)/\tau)$, with τ a "temperature" parameter that controls the randomness [8]. A high τ produces near-uniform random behavior (high exploration), while a low τ

makes the policy greedier. Another approach is optimistic initial values: initialize $Q(s, a)$ optimistically high so that the agent is initially drawn to explore each action to discover that its value is (perhaps) lower than anticipated [2]. More sophisticated methods use uncertainty estimates or intrinsic bonuses (as in upper-confidence-bound algorithms for bandits or Bayesian RL), but ϵ -greedy remains a simple and widely used method due to its ease of implementation and empirical effectiveness.

Maintaining sufficient exploration is crucial in single-agent RL to ensure the agent does not get trapped in suboptimal actions. In multi-agent settings, exploration becomes even more complex, as an agent’s exploratory actions also affect the learning of other agents and vice versa, sometimes necessitating coordinated exploration or experimentation. Nonetheless, the basic principle remains: long-term optimal performance requires occasionally trying out actions that might seem suboptimal in the short term in order to discover better strategies.

2.4 Reward Structures and Shaping

The design of the reward function greatly influences the learning process in reinforcement learning. In an MDP or multi-agent game, the reward structure can be sparse (only occasional non-zero rewards, e.g., success/failure signals) or dense (frequent feedback for intermediate steps), and aligned or conflicting across agents (in multi-agent scenarios). A well-shaped reward can guide an agent more effectively toward desired behaviors, whereas a poorly designed reward can lead to slow learning or unintended behaviors as the agent tries to game the reward signal.

Reward shaping is the practice of augmenting or modifying the reward function to make learning easier without changing the underlying optimal policy of the task. One classic result in this area is by Ng *et al.* (1999), who introduced the concept of potential-based reward shaping and proved that adding a shaping reward of a specific form preserves the optimal policy [9]. In potential-based shaping, one defines a potential function $\Phi : S \rightarrow \mathbb{R}$ over states, and adds an auxiliary reward defined as the difference in potential between consecutive states. For example, the shaping reward can be $F(s, s') = \gamma\Phi(s') - \Phi(s)$, so that the total reward becomes $R'(s, a, s') = R(s, a, s') + F(s, s')$ [9]. This shaping reward is defined in such a way that it is always the difference of a potential, ensuring that for any closed loop of state transitions the sum of shaping rewards is zero. Ng *et al.* proved that this kind of reward shaping does not alter the set of optimal policies of the MDP, but it can substantially speed up learning by guiding the agent with additional hints. Intuitively, $\Phi(s)$ can be seen as an encoding of domain knowledge (heuristic) about the desirability of states, and the shaping reward provides a graded incentive for the agent to move toward states of higher potential.

In multi-agent systems, reward structures can be cooperative or competitive as discussed earlier, and shaping can also be applied. For fully cooperative teams, one can shape the common reward to ease credit assignment or to encourage exploration of beneficial behaviors that are rare with sparse team rewards. In competitive scenarios, one has to be careful: shaping one agent’s reward might effectively disadvantage another if not done symmetrically, though potential-based shaping can in theory be applied individually for each agent’s reward function if it does not distort the equilibrium.

Another aspect of reward design is avoiding perverse incentives or undesired solutions. A reinforcement learner will exploit any loophole in the reward definition. Thus, shaped rewards should be designed to truly reflect intended goals. A common recommendation is to start with the simplest, most directly aligned reward with the task objective (e.g., +1 for goal success, 0 otherwise) and only add shaping terms if the agent struggles to learn, all while ensuring that the optimal policy under the shaped reward is still an optimal (or near-optimal) policy for the original task.

Reward shaping is a powerful tool but is also problem-specific; it requires intuition or knowledge of the domain. When done correctly, however, it can dramatically improve learning speed. For example, in navigation tasks, giving a small negative reward for each time step encourages quicker solutions (time-based shaping), and giving a bonus for moving closer to the goal (distance-based potential shaping) can guide exploration in large mazes. Empirical and theoretical studies have reinforced that shaping, especially potential-based shaping, can be integrated into Q-learning or other algorithms without breaking convergence guarantees, provided the shaping is carefully constructed [9].

2.5 Minimax-Q Learning

Minimax-Q is an algorithm designed for multi-agent reinforcement learning in two-player zero-sum Markov games, introduced by Littman [4]. In a zero-sum setting, the interests of two agents are strictly opposed; one agent (the “maximizing” player) seeks to maximize the return, while the other (the “minimizing” player or adversary) seeks to minimize the first agent’s return. A solution concept for such games is the minimax value, which extends the idea of a saddle-point equilibrium from static zero-sum games to the sequential (Markov) setting.

In a Markov game with two players, states $s \in S$, actions $a^1 \in A^1$ for the agent and $a^2 \in A^2$ for the adversary, let $Q(s, a^1, a^2)$ denote the Q-value for the agent (player 1) for state s and joint action (a^1, a^2) . In a zero-sum game, the adversary’s reward is $-Q$ (if we assume Q already represents the payoff to player 1). In Minimax-Q learning, the agent updates its Q-values by assuming the adversary will play optimally to minimize the agent’s payoff. Thus, instead of using a $\max_{a'} Q(s', a')$ term as in standard Q-learning, the agent uses a min-max value of the next state. Formally, the value of a state s (from the agent’s perspective) is $V(s) = \max_{a^1 \in A^1} \min_{a^2 \in A^2} Q(s, a^1, a^2)$, which is the game-theoretic value if both players play optimally from state s [4]. Let a^{1*} and a^{2*} be the actions achieving this saddle point (in general, there might not be a pure-action saddle point; in that case a mixed strategy equilibrium is considered). The Minimax-Q update rule can be written as:

$$Q(s, a^1, a^2) \leftarrow Q(s, a^1, a^2) + \alpha \left[r^1(s, a^1, a^2) + \gamma V(s') - Q(s, a^1, a^2) \right], \quad (2.3)$$

where $r^1(s, a^1, a^2)$ is the reward to the agent (player 1) in state s under the joint action (a^1, a^2) , and $V(s') = \max_{a^1} \min_{a^2} Q(s', a^1, a^2)$ as defined above for the next state s' [4]. The agent thus incorporates the assumption that the opponent will choose a^2 to minimize $Q(s', \cdot, a^2)$ when deciding the target value.

In practice, computing $\min_{a^2} Q(s', a^1, a^2)$ for each a^1 and then the max over a^1 can be done by solving a small two-player zero-sum game (the “stage game” at state s') defined by payoff matrix $Q(s', a^1, a^2)$ for player 1. If the action sets are small, this can be done via

brute force or linear programming (for larger action spaces). Littman’s original formulation handles the case of possibly mixed optimal strategies by incorporating a linear programming step to compute the state values if needed [4].

Minimax-Q learning is proven to converge to the optimal value function for two-player zero-sum stochastic games under similar conditions required for Q-learning (infinitely many state-action visits, decaying learning rates, etc.), assuming both players either follow the learning algorithm or the opponent follows some stationary (possibly adversarial) policy that at least occasionally allows exploration [4]. A later analysis by Littman and Szepesvári (1996) provided a convergence proof using game-theoretic and dynamic programming techniques to show that the Minimax-Q update finds the correct value function for zero-sum games, which corresponds to the unique Nash equilibrium (value) in such games. Intuitively, because zero-sum games have the property that optimal play leads to a single value (the game value) for each state, the learning process can home in on those values even as the two players continually adapt.

Minimax-Q is a cornerstone for more advanced multi-agent RL algorithms in competitive settings. It essentially integrates a simple game solver (for the immediate rewards plus estimated future values) into the temporal-difference update. One limitation is that it assumes knowledge of the actions the opponent takes (since it updates on a specific (a^1, a^2) pair and requires evaluating the min-max over opponent actions). In self-play scenarios (where both agents use Minimax-Q), each agent would be computing its value assuming the other is adversarial. If both agents follow Minimax-Q and adequately explore, they should in principle converge to a Nash equilibrium of the zero-sum game. If the opponent’s policy is not actually adversarial (e.g., a fixed policy or some other behavior), a Minimax-Q learner is essentially learning a cautious policy that guards against worst-case outcomes, which might be overly conservative. Nonetheless, in truly competitive tasks, Minimax-Q provides a learning rule aligned with robust optimal play.

2.6 Nash-Q Learning

In general-sum stochastic games (where agents’ interests are neither fully aligned nor completely opposed), the concept of Nash equilibrium can be used as a solution criterion. Nash-Q learning is an extension of Q-learning to multi-agent general-sum games, developed by Hu and Wellman [5]. The key idea is that, at each state, instead of taking a max over actions (single-agent) or a min-max (two-player zero-sum), the learning rule uses the solution of a game (a Nash equilibrium) defined by the Q-values as the payoff matrix for that state.

Consider, for simplicity, a two-agent general-sum Markov game with agents 1 and 2. Let $Q^1(s, a^1, a^2)$ and $Q^2(s, a^1, a^2)$ denote the Q-values for each agent at state s for the joint action (a^1, a^2) . A Nash equilibrium at state s is a pair of strategies (one for each agent) (π^1, π^2) over their action sets A^1, A^2 such that π^1 is a best response to π^2 given payoffs $Q^1(s, \cdot, \cdot)$ and π^2 is a best response to π^1 given payoffs $Q^2(s, \cdot, \cdot)$. Let $\text{NashEq}_i(Q^1(s), Q^2(s))$ denote the value to agent i under some (e.g., symmetric) Nash equilibrium of the one-stage game with payoff matrices $Q^1(s)$ and $Q^2(s)$. Nash-Q learning updates Q-values in a manner coupled with these equilibrium computations:

$$Q^i(s, a^1, a^2) \leftarrow Q^i(s, a^1, a^2) + \alpha \left[r^i(s, a^1, a^2) + \gamma V^i(s') - Q^i(s, a^1, a^2) \right], \quad (2.4)$$

where $V^i(s') = \text{NashEq}_i(Q^1(s'), Q^2(s'))$ is the value for agent i at the next state s' assuming both agents play a Nash equilibrium strategy in s' [5]. Thus, each agent's update uses as the future value the expected payoff it would get if both agents act according to a Nash equilibrium at the next state.

In practice, at each state s' the algorithm needs to compute a Nash equilibrium of the stage game defined by the current $Q^1(s')$ and $Q^2(s')$ matrices. For two players, this is a bimatrix game, and one can use linear programming or Lemke-Howson algorithm to find a Nash equilibrium (for general-sum 2-player games, there may be multiple Nash equilibria; Hu and Wellman focus on learning with one equilibrium concept, and typically assume that a particular equilibrium selection is applied consistently). In their original formulation, Hu and Wellman assume that the game at each state has a unique Nash equilibrium in mixed strategies (or that players can coordinate on one equilibrium), which is a strong assumption but necessary for stable learning. If there are multiple equilibria, the learning can become unstable or ill-defined unless an equilibrium selection scheme is built in.

Hu and Wellman proved that if each stage game has a unique Nash equilibrium that is consistently selected, and certain conditions hold (they give assumptions ensuring the learning algorithms' iterates remain within a set supporting that equilibrium), then the Nash-Q algorithm will converge to a set of Q-values corresponding to a Nash equilibrium policy for the whole game [5]. In essence, Nash-Q attempts to learn the equilibrium of the multi-agent game by iteratively enforcing equilibrium play at each state in the future.

However, in general-sum games, learning a Nash equilibrium is significantly harder than learning a single-agent optimum. One challenge is the equilibrium selection problem: which equilibrium should the agents converge to if multiple exist? Another challenge is that each agent learning simultaneously can lead to non-stationary targets. Bowling and Veloso (2000) noted that algorithms like Nash-Q and Minimax-Q aim to converge to equilibrium policies regardless of the opponents' actual behaviors, but in practice they can oscillate or fail if the assumptions (like uniqueness and full rationality) are not met. Indeed, Nash-Q may diverge or cycle in some games, or converge to a suboptimal equilibrium, if, for example, agents follow different equilibrium selection conventions or if the game payoff structure leads to stochastic switching between equilibria.

Despite these difficulties, Nash-Q learning was an important early attempt to generalize Q-learning to multi-agent settings with strategic interactions [5]. It paved the way for further research into equilibrium-based learning (such as Friend-or-Foe Q-learning for specific classes of games, or more recent ideas combining policy gradient with game-theoretic solution concepts). In summary, Nash-Q provides a framework where each agent, in effect, pretends that at each future state all agents will play an equilibrium strategy, and updates its values accordingly. If the environment and other agents indeed align with those expectations (i.e., all converge to equilibrium strategies), the algorithm can find equilibrium policies. If not, Nash-Q may struggle, highlighting the inherent complexity of general-sum multi-agent learning.

2.7 Dyna-Q: Integrating Learning and Planning

Many reinforcement learning algorithms, including vanilla Q-learning, are model-free, meaning they do not attempt to learn the transition probabilities or reward function explicitly, but rather learn value functions or policies directly from experience. In contrast, model-

based approaches involve learning a model of the environment (transition and reward function) and then planning with that model to improve policy or value estimates. The Dyna framework, introduced by Sutton [10], is a hybrid approach that integrates direct learning with planning. The idea is that an agent can use its experiences to both update its value function (learning) and also to build a model from which it can simulate additional experiences (planning), thereby accelerating learning by rehearsing or extrapolating from what it has observed.

In the simplest instantiation, Dyna-Q, the agent maintains an approximate model $\hat{P}(s'|s, a)$ and $\hat{R}(s, a)$ alongside the Q-value table. Whenever the agent experiences a transition (s, a, r, s') , it performs two updates:

1. Direct reinforcement learning update: It updates $Q(s, a)$ toward $r + \gamma \max_{a'} Q(s', a')$ as in normal Q-learning (Equation 2.2).
2. Model update: It updates its model: e.g., record that from s taking a led to s' and reward r . In a simple tabular model, this could be storing $(s, a) \mapsto (s', r)$ if the environment is deterministic; in a stochastic setting, it might update a distribution or counts for transitions from (s, a) .
3. Planning updates: The agent then conducts N simulated updates by sampling from its model. For instance, it might randomly pick N state-action pairs $(\tilde{s}, \tilde{a})$ that it has seen before, query its model to get a predicted outcome $(\tilde{s}', \tilde{r}) = \hat{P}, \hat{R}(\tilde{s}, \tilde{a})$, and then perform a Q-learning update $Q(\tilde{s}, \tilde{a}) \leftarrow Q(\tilde{s}, \tilde{a}) + \alpha[\tilde{r} + \gamma \max_{a'} Q(\tilde{s}', a') - Q(\tilde{s}, \tilde{a})]$ using the simulated experience [2, 10]. These planning updates allow the agent to internally rehearse experiences and propagate information more quickly to relevant states.

By intermixing real experience with model-based planning, Dyna-Q can significantly speed up learning. It effectively performs incremental Dynamic Programming updates using the learned model. Sutton's experiments showed that even a modest number of planning steps N per real step can yield substantial improvements [10]. Notably, Dyna-Q does not require a perfect model; it continually refines the model as new data arrives, and planning uses the latest model. This makes Dyna suitable for non-static environments as well, to some extent, since the model and values can adapt.

2.7.1 Dyna-Q+ (Dynamic Environments and Exploration Bonus)

Dyna-Q+ is an extension of Dyna-Q that is designed to handle changing environments and to encourage continued exploration [2]. In a changing or non-stationary environment, an agent should not fully trust an outdated model; it needs to keep exploring to detect changes (for example, if certain transitions or rewards change after some time). Dyna-Q+ introduces two modifications:

1. It keeps track of the time elapsed since each state-action pair was last tried.
2. It adds an exploration bonus to the reward of state-action pairs in planning updates that have not been tried for a long time.

Concretely, in Dyna-Q+, the model might be augmented such that for a state-action (s, a) , if it has not been visited for τ time steps, the planning query for (s, a) returns

a “fictitious” reward $\tilde{r} = \hat{R}(s, a) + k\sqrt{\tau}$ (one example functional form of bonus) or some bonus $+\kappa$ after a certain time gap, where k or κ is a small positive constant [10]. This bonus reward in simulation means that the planner will treat unvisited (or long-unvisited) state-actions as potentially more valuable, driving the Q-values of those state-actions upward, which in turn encourages the real agent to eventually try them (since they look promising). The net effect is that Dyna-Q+ agents continue to explore new or unvisited actions even after a seemingly optimal policy has been found, thereby being prepared to quickly adapt if the environment changes (e.g., a previously suboptimal action might become optimal if the environment’s dynamics shifted).

Sutton [10] demonstrated the advantage of Dyna-Q+ in a “blocking” maze task: initially a shortcut may be blocked, and later the block is removed. A standard Dyna agent might stick with the long route it learned first, because it has no incentive to re-explore the blocked path. Dyna-Q+, however, by virtue of the exploration bonus, will eventually retry that path, discover the change, and rapidly adjust to the new optimal route. In static environments, Dyna-Q+ may perform slightly worse than Dyna-Q (since it wastes some time exploring seemingly better actions that actually are not better), but in dynamic environments it can be far superior [2]. The exploration bonus essentially ensures the agent doesn’t converge prematurely when the environment might still have surprises.

2.8 Deep Q-Networks (DQN)

Tabular Q-learning becomes impractical when the state space grows large, since it requires explicitly storing a value for every state–action pair. Deep Q-Networks (DQN) address this by using a neural network $Q(s, a; \theta)$ as a function approximator for the action-value function. Instead of a table, the network parameters θ are optimized so that the predicted Q-values satisfy the Bellman optimality relation. DQN is trained by minimizing the squared temporal-difference (TD) error over transitions (s, a, r, s') sampled from an experience replay buffer:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (2.5)$$

where θ^- denotes the parameters of a separate *target network* that is updated periodically to stabilize learning. Experience replay further improves stability by breaking correlations in sequential data and enabling more efficient reuse of past experience. In this project, we implemented a lightweight DQN agent as an initial step toward function approximation beyond the tabular setting, while retaining ϵ -greedy exploration for action selection [11].

Chapter 3

Problem Setup and Evaluation Protocol

3.1 Gridworld Environment Overview

We consider two multi-agent environments – a basic pursuit scenario (*Chase*) and an extended variant (*Wumpus Chase*) – both implemented as gridworld games. The environment can be formalized as a two-player Markov game [4], where at each discrete time step both agents observe the current state and choose actions simultaneously. The state is typically defined by the positions of all agents (and any important objects) on an $N \times N$ grid. An action for an agent is a movement in one of the four cardinal directions or a **STAY** action (no movement). All chosen actions are executed concurrently in the environment. Movement actions are bounded by the grid limits and may be blocked by impassable cells (obstacles) if present. We introduce a small probability of failure p_{fail} for each action: with probability p_{fail} , an agent’s intended move does not occur (the agent remains in the same cell). This stochastic element models real-world uncertainty and makes the environment non-deterministic.

Each episode begins by placing the agents at random starting positions on the grid (typically in empty cells without any special items or hazards). The agents then take actions simultaneously at each step and the environment transitions accordingly. The episode continues until a terminal condition is reached. In general, an episode will terminate when any of the following conditions occurs:

- Capture: One agent “tags” or lands on the same cell as the other (or they cross paths in a single time step), indicating a capture event.
- Goal achievement: An agent reaches a designated goal location (if such a goal exists in the scenario). For example, in the Wumpus Chase scenario there is a *treasure* cell; if an agent reaches the treasure first, the episode ends with that agent achieving the goal.
- Hazard trigger: An agent steps into a lethal hazard cell, such as the *Wumpus* in the Wumpus Chase scenario. This results in an immediate loss for the agent that triggered the hazard and terminates the episode.

- Time limit: A maximum number of time steps $t_{\max}$ is reached without any of the above events. In this case the episode terminates due to timeout (typically considered a draw or inconclusive outcome).

These rules apply to both the Chase and Wumpus Chase scenarios, with the difference that the Wumpus Chase includes additional goal and hazard termination conditions (treasure capture or Wumpus death) not present in the basic Chase scenario. In all cases, both agents move simultaneously at each step and thus have equal opportunity to act, making the interaction a synchronous move game. This avoids turn-taking biases and means that, for example, a capture can occur by both agents moving into the same cell or swapping positions on the same step. After termination, the environment is reset for the next episode. The grid layout (including any obstacles, hazard, or goal locations) remains consistent across episodes, but the starting positions of agents are randomized to encourage robustness across different initial configurations.

3.2 Reward Structure

The reward function in these environments is designed to encode the objectives of each agent and to facilitate effective learning through reward shaping. At a high level, the tasks are competitive: one agent’s gain is often the other’s loss (particularly for win/lose outcomes). Major events in the game yield significant rewards or penalties to signal success or failure, while additional small rewards are given at each step to guide behavior. We summarize the reward structure generally as follows:

- Outcome Rewards: A large positive reward is given to an agent that achieves a terminal objective (such as capturing the opponent or reaching the treasure), while an equally large negative reward is given to the opposing agent. For example, if Agent A captures Agent B, Agent A might receive $+R$ points and Agent B receives $-R$ (with R a high value). Similarly, reaching the goal or surviving until timeout can be considered a success for one agent and failure for the other, in which case rewards are assigned accordingly. These outcome rewards ensure that winning the game is the most strongly incentivized event.
- Hazard Penalty: If an agent triggers a hazard (like falling into the Wumpus cell), that agent incurs a large negative reward (and typically the other agent gains a corresponding positive reward by essentially “winning” as a result). This discourages stepping into fatal cells. Hazards lead to immediate termination, so this is effectively another outcome reward/penalty scenario.
- Time Step Penalty: To encourage agents to finish the task quickly, a small penalty is applied for each time step that the game continues. In other words, living too long without a resolution slightly diminishes the reward. This per-step cost (for example, -0.01 each step) motivates agents to capture or achieve goals faster, rather than stalling. Depending on the scenario design, this penalty might be applied symmetrically or only to specific agents (e.g., only the pursuer in a chase scenario might get the time penalty).

- **Movement and Action Rewards:** We often include a minor reward for simply executing a valid move (e.g. a small positive reward for moving to a new cell) and possibly a small penalty for invalid actions (such as attempting to move into a wall or obstacle). For instance, if an agent tries to move out of bounds or into an obstacle, the move is blocked; a slight negative reward can be given to reinforce that this action is undesirable. Conversely, actually moving (not staying idle) could yield a tiny positive reward to encourage exploration and activity.
- **Distance-Based Shaping:** We incorporate distance-based shaping rewards to guide pursuit and evasion behaviors. At each step, the change in distance between agents (and/or between an agent and a goal) yields a small reward or penalty. For example, if the distance between a pursuer and the evader decreases, the pursuer gets a small positive reward (and the evader a corresponding small negative reward), since the pursuer is closer to catching its target. If the distance increases, the pursuer could receive a negative reward while the evading agent gains a positive reward for successfully increasing their distance. In the Wumpus Chase scenario, a similar idea applies to the treasure: an agent may receive a small positive reward for moving closer to the treasure, and possibly a negative reward if the opponent moves closer to the treasure. These shaping rewards are carefully chosen to align with the ultimate goals (capture or goal achievement) so that they provide hints to the agent about what behaviors are productive, without outweighing the primary outcome rewards.

In summary, the reward structure provides a significant positive/negative signal for winning or losing events and augments this with minor step-wise rewards to encourage effective strategies (this technique is known as reward shaping [9]). The overall game is essentially zero-sum at the level of outcomes (one agent’s win is the other’s loss), but due to shaping terms the reward function at intermediate time steps is not strictly zero-sum. Nonetheless, each agent’s goal in training is to maximize its own cumulative reward, which, given the competitive setup, indirectly means outplaying the opponent. By receiving frequent small feedback signals (e.g., distance changes), the agents can learn more smoothly rather than only from sparse terminal rewards.

3.3 Training Protocol

We train the agents using a multi-agent Q -learning approach over many repeated episodes of interaction. Training is episodic: the agents play a large number of episodes in succession, improving their policies over time through repeated experience. Algorithm 1 outlines the general training loop employed for both scenarios.

During training, ϵ -greedy exploration is crucial to ensure the agents adequately explore the state-action space. We typically start with a relatively high ϵ (such as 0.2 or 0.3 in our experiments) to encourage exploration of random actions, and then gradually reduce ϵ over the course of training to a lower value (e.g., 0.05). This schedule allows the agents to explore widely in early training (preventing premature convergence to a suboptimal policy) and to exploit accumulated knowledge in later stages once the environment dynamics are better understood. The decay can be linear or exponential over a predefined number of episodes. By the end of training, the policy is nearly greedy (almost always taking the best-known action), which typically yields the best performance after sufficient learning.

Algorithm 1: Episodic Multi-Agent Training Loop

1. **Initialize:** Set episode count = 0. Initialize agents' Q -tables or function approximators to initial values. Set the exploration rate ϵ (for ϵ -greedy) to its starting value ϵ_{start} .
 2. **Loop over episodes:** For each episode $i = 1, 2, \dots, N_{\text{episodes}}$:
 - (a) Reset the environment to a start state (randomize agent positions, etc.).
 - (b) For time step $t = 1, 2, \dots$ until episode termination:
 - i. Each agent observes the current state s_t .
 - ii. Each agent selects an action a_t^A (Agent A) and a_t^B (Agent B) according to its current policy **with exploration**. In practice, we use an ϵ -greedy policy [2]: with probability ϵ an agent chooses a random action (exploration), and with probability $1 - \epsilon$ it chooses the greedy action (the action with highest Q -value for the current state).
 - iii. Execute both agents' actions simultaneously in the environment. The environment returns the next state s_{t+1} , a reward r_t^A for Agent A, a termination flag **done** $\in \{\text{true}, \text{false}\}$, and an outcome label if **done**. (In our implementation, the runner's reward r_t^B is available via the environment **info** dictionary when needed.)
 - iv. Each agent updates its value estimates based on the transition. For example, a standard Q -learning update is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right],$$
 where α is the learning rate and γ is the discount factor [2]. In the multi-agent setting, Agent A and Agent B may update their own value functions (or joint-action Q tables) simultaneously using their respective rewards.
 - v. Log the transition and any relevant statistics (detailed in Section 3.4).
 - vi. If **done** is **true**, break out of the time-step loop.
 - (c) End of episode: increment episode counter. Optionally adjust the exploration rate ϵ according to a decay schedule (e.g., linearly decrease ϵ after each episode down to a minimum ϵ_{end}).
 - (d) If the current episode index i is a multiple of the evaluation interval E (e.g., every 100 episodes), pause training and perform an evaluation (described in Section 3.3).
-

The training process runs for a fixed number of episodes N_{episodes} (for example, tens of thousands of episodes). This number is chosen large enough to allow convergence of the Q -learning algorithm given the complexity of the environment. Throughout training, we maintain a log of important data from each episode (e.g., cumulative rewards, whether the agent succeeded, etc.) and from each time step (state transitions and rewards). The precise logging strategy and data recorded are discussed in Section 3.4.

3.3.1 Periodic Evaluation Protocol

To monitor learning progress and evaluate the quality of the policies being learned, we intersperse regular evaluation phases into the training process. Every E episodes of training (with E set to a value like 100), the training is temporarily halted and a batch of evaluation episodes is run using the current policies. During these evaluation episodes, exploration is turned off (we set $\epsilon = 0$) so that each agent follows a purely greedy policy derived from its current Q -values. In other words, the agents exploit what they have learned so far, without any random exploratory moves, to see how well the current policy would perform in a consistent deployment setting.

Each evaluation phase consists of a fixed number of episodes (for instance, 10) played with the current policy. The environment is re-initialized for these episodes just as in training (often using a separate random seed or a deterministic sequence of test scenarios to ensure fairness and coverage). We record performance metrics during evaluation, such as the win rate of each agent (percentage of episodes won by each side), the average episode length, and the average cumulative reward achieved. Because no learning updates are performed during evaluation, these episodes do not alter the Q -values; they strictly provide a measurement of performance. After the evaluation episodes are completed, training resumes from where it left off.

Including periodic evaluations allows us to produce a learning curve over the course of training, typically plotting metrics like win rate or average reward vs. training episodes. It provides an unbiased check on how the policy is improving, since during training episodes the ongoing exploration can make the raw outcomes noisy or optimistic/pessimistic. By evaluating with $\epsilon = 0$, we get a clearer picture of the policy’s true capabilities at that point in time. If desired, the evaluation frequency E can be adjusted (smaller E for more frequent evaluations to get a fine-grained view, or larger E for less frequent testing to save time).

For reliable comparison, the evaluation uses a fixed protocol each time. For example, we may use a fixed set of evaluation random seeds offset from the training seeds (e.g., using seed values in a different range) to ensure that evaluation encounters a variety of starting configurations and is not inadvertently testing on the same sequence of randomness that training saw. This helps in assessing generalization. The results of each evaluation phase are logged to a separate file (or in memory) for later analysis (e.g., a CSV or spreadsheet of evaluation outcomes at each checkpoint).

3.4 Logging and Experimental Settings

Throughout both training and evaluation, the system implements a comprehensive logging scheme to record data for analysis and to ensure reproducibility of results. In this section

we outline what data is collected and the common experimental parameters used across runs.

3.4.1 Data Logging Scheme

Every episode generates a wealth of information that can be logged. We log at two levels of granularity: *step-level* (each time step within an episode) and *episode-level* summary. At the step-level, for each transition $(s_t, a_t^A, a_t^B, r_t^A, r_t^B, s_{t+1})$, we record the full state of the environment before the action, the joint actions taken by both agents, and the immediate rewards received by each agent. The next state s_{t+1} is also recorded, which effectively gives us a sequence of state-action-reward-state (SARS) tuples for every step. In addition to these core fields, we log extra metadata that provides context for the transition. This metadata can include: whether a capture occurred on that step, whether either agent’s action failed due to p_{fail} , the distance between agents (before and after the move), indicators of specific events like “agent stepped into hazard” or “agent picked up the treasure”, and the time step index t . We also record the individual components of the reward (for debugging and analysis of the shaping terms), such as how much of the reward was due to outcome, how much due to distance change, step penalty, etc. (these were outlined in the reward structure and are obtainable from the environment info). This fine-grained log is invaluable for diagnosing the agents’ behaviors and understanding why certain actions were taken.

At the end of each episode, we log summary statistics for that episode. Key episode-level data includes: which agent (if any) won the episode, the terminal condition or outcome type (e.g., “A captured B”, “B captured A”, “A fell into Wumpus”, “timeout/draw”), the number of steps the episode lasted, and the total cumulative reward obtained by each agent over the episode. We also log the episode index and the random seed used for that episode’s environment initialization (to allow replaying it if needed). During evaluation phases, we similarly log outcomes and possibly the full trajectories as well (though sometimes only summary stats are recorded for evaluation episodes, since no learning happens during them). All logs are written to structured files (such as CSV spreadsheets or databases), with separate files or tabs for training episodes and evaluation checkpoints. This organization makes it easier to analyze training progress and performance trends after the experiments.

3.4.2 Experimental Parameters and Reproducibility

We ensure that all experiments share a common set of configuration parameters, and we document these for clarity. Table 3.1 lists some of the key shared settings for the experiments in both Chase and Wumpus Chase scenarios.

These values are kept constant unless otherwise specified, to allow a fair comparison between different algorithms or agent configurations in later chapters. Notably, the probability of action failure $p_{\text{fail}} = 0.1$ introduces stochastic dynamics uniformly in both scenarios. The maximum steps t_{max} is chosen based on grid size and game difficulty to give agents sufficient time (e.g., up to 30 or 40 moves) to accomplish their goals; if this limit is reached, the episode is deemed a draw. The number of training episodes is on the order of tens of thousands, which was found to be sufficient for learning stable policies in these gridworlds. Evaluation is frequent enough to track progress (every few dozen to hundred episodes) but infrequent enough not to dominate runtime.

Table 3.1: Common Environment and Training Parameters

Grid size	5×5 in Chase (basic tag), 7×7 in Wumpus Chase (extended)
Max episode length $t_{\max}$	30 steps (Chase), 40 steps (Wumpus Chase)
Action failure probability p_{fail}	0.10 (10% chance each action fails to execute)
Training episodes N_{episodes}	$\sim 10^4$ – 3×10^4 episodes (long enough for convergence)
Evaluation frequency E	Every 50 episodes (Chase) / 100 episodes (Wumpus)
Evaluation episodes	10 episodes per evaluation checkpoint (default)
Exploration (ϵ -greedy)	$\epsilon_{\text{start}} = 0.2$ (Chase) / 0.3 (Wumpus), decaying to $\epsilon_{\text{end}} = 0.05$
Discount factor γ	0.95 (for Q -learning updates in all cases)
Learning rate α	0.10 (baseline value for tabular Q updates)
Random seed strategy	Fixed base random seed per run, evaluation uses offset seeds to avoid overlap

To ensure reproducibility of results, we fix random seeds for each experimental run. The random seed influences the environment’s random features (like starting positions and action failure outcomes) as well as any randomness in agent learning updates. By using the same seed, we can replay the training and get identical behavior, which is crucial for debugging and verifying results. When conducting multiple independent runs (to assess variability), we simply use different seed values for each run. Furthermore, as mentioned, evaluation phases often use a different sequence of random seeds (for example, $\text{seed} = \text{base_seed} + 10000 + \text{episode_index}$) to ensure that the evaluation scenarios are not exactly the same as those seen during training. This separation between training and evaluation randomness helps test the generalization of learned policies. All important configuration parameters (like those in Table 3.1) are logged and documented alongside the results, so that experiments are fully transparent and can be replicated by other researchers.

In conclusion, this chapter has detailed the common environment setup and experimental protocol for our multi-agent Q -learning study. We described the gridworld scenarios (Chase and Wumpus Chase) in which the agents operate, the design of the reward signal that guides learning, and the training regimen including exploration and periodic evaluation. We also outlined how we log data and maintain consistent settings for reproducibility. The subsequent chapters will build on this foundation, diving into scenario-specific details and the performance of various learning algorithms under this protocol.

Chapter 4

Base Chasing

4.1 Scenario Overview

The *Chase* scenario is a two-player simultaneous-move gridworld pursuit–evasion game. Agent A (the *Catcher*) aims to capture Agent B (the *Runner*) as quickly as possible, while Agent B aims to avoid capture until the episode terminates. The environment is intentionally minimal, yet it retains key multi-agent reinforcement learning (MARL) difficulties: simultaneous interaction, stochastic transitions, and reward shaping that encourages meaningful intermediate behavior rather than relying only on sparse terminal signals.

4.2 Environment Specification (TagEnv)

4.2.1 Grid, Coordinates, and State Space

The environment is defined on an $N \times N$ grid with default size $N = 5$. Coordinates are represented as (x, y) pairs, where x corresponds to the row index and y to the column index. The state at time t is the joint-position tuple

$$s_t = (x_t^C, y_t^C, x_t^R, y_t^R), \quad (4.1)$$

where (x_t^C, y_t^C) denotes the Catcher position and (x_t^R, y_t^R) the Runner position. Under this representation, the discrete state space cardinality is $(N^2) \cdot (N^2)$ (for $N = 5$, this yields $25 \cdot 25 = 625$ states).

4.2.2 Action Space and Simultaneous Dynamics

Both agents act simultaneously at each time step using the same discrete action set

$$\mathcal{A} = \{\text{STAY}, \text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}, \quad (4.2)$$

with the corresponding movement deltas

$$\Delta(\text{STAY}) = (0, 0), \Delta(\text{UP}) = (+1, 0), \Delta(\text{DOWN}) = (-1, 0), \Delta(\text{LEFT}) = (0, -1), \Delta(\text{RIGHT}) = (0, +1).$$

Movements are clamped to remain inside the grid. Let $\text{clip}(\cdot)$ denote component-wise clipping into $\{0, 1, \dots, N - 1\}$. If an agent at (x, y) intends to move by (δ_x, δ_y) , the candidate next position is

$$(x', y') = (\text{clip}(x + \delta_x), \text{clip}(y + \delta_y)). \quad (4.3)$$

Importantly, attempting to move outside the grid results in no change of position due to clipping, but the action is still recorded as a wall-hit event for reward penalties (Section 4.3.1).

4.2.3 Stochastic Action Failure

To introduce stochasticity, each agent independently experiences action failure with probability p_{fail} (default $p_{\text{fail}} = 0.10$). If failure occurs, the agent remains in its current cell regardless of its chosen action. This produces non-deterministic transitions and prevents policies from relying on perfectly predictable movement outcomes.

4.2.4 Capture Conditions

The episode is considered *captured* if either of the following holds after applying both actions (and any failures):

1. **Same-cell capture:** both agents occupy the same cell, i.e. $(x_{t+1}^C, y_{t+1}^C) = (x_{t+1}^R, y_{t+1}^R)$.
2. **Cross-path capture:** the agents swap positions in one step, i.e. $(x_{t+1}^C, y_{t+1}^C) = (x_t^R, y_t^R)$ and $(x_{t+1}^R, y_{t+1}^R) = (x_t^C, y_t^C)$.

The cross-path rule ensures that a direct “pass-through” is treated as a capture event even if the agents do not end the step in the same cell.

4.2.5 Episode Termination

Let t_{max} denote the time limit (default $t_{\text{max}} = 30$). An episode terminates if:

- a capture event occurs (Section 4.2.4), or
- the time counter reaches the maximum horizon ($t \geq t_{\text{max}}$), producing a timeout.

Timeouts are treated as “no-capture” outcomes and are reflected in evaluation metrics such as capture rate and steps-to-capture (Chapter ??).

4.3 Reward Design and Shaping

4.3.1 Decomposed Reward Components

The environment implements a shaped reward signal designed to provide both terminal outcome feedback and dense intermediate guidance. While the `step` interface returns only the Catcher reward, the Runner reward is computed simultaneously and exposed through an auxiliary `info` structure for learning and logging.

Default shaping parameters are:

$$R_{\text{cap}} = 10.0, \quad \lambda_{\text{step}} = 0.01, \quad \lambda_{\text{wall}} = 0.02, \quad \rho_{\text{move}} = 0.01, \quad \rho_{\text{dist}} = 0.02.$$

Let $\mathbb{I}_{\text{cap}}$ indicate capture at time t . Let $\mathbb{I}_{\text{wall}}^i$ indicate whether agent $i \in \{C, R\}$ attempted an out-of-bounds move. Let $\mathbb{I}_{\text{move}}^i$ indicate whether agent i actually changed cell. Let d_t be the Manhattan distance between agents at time t :

$$d_t = |x_t^C - x_t^R| + |y_t^C - y_t^R|. \quad (4.4)$$

The distance-shaping term depends on $\Delta d_t = d_{t+1} - d_t$.

The Catcher reward is the sum of:

$$r_t^C = \underbrace{\mathbb{I}_{\text{cap}} R_{\text{cap}}}_{\text{capture}} + \underbrace{(1 - \mathbb{I}_{\text{cap}})(-\lambda_{\text{step}})}_{\text{step penalty (C only)}} + \underbrace{(-\lambda_{\text{wall}}) \mathbb{I}_{\text{wall}}^C}_{\text{wall penalty}} + \underbrace{\rho_{\text{move}} \mathbb{I}_{\text{move}}^C}_{\text{move reward}} + \underbrace{r_{\text{dist}}^C(\Delta d_t)}_{\text{distance shaping}}, \quad (4.5)$$

where

$$r_{\text{dist}}^C(\Delta d_t) = \begin{cases} +\rho_{\text{dist}}, & \Delta d_t < 0, \\ -\rho_{\text{dist}}, & \Delta d_t > 0, \\ 0, & \Delta d_t = 0. \end{cases}$$

The Runner reward mirrors the competitive components (capture and distance) but retains shared movement and wall terms:

$$r_t^R = \underbrace{-\mathbb{I}_{\text{cap}} R_{\text{cap}}}_{\text{capture (opposite sign)}} + \underbrace{(-\lambda_{\text{wall}}) \mathbb{I}_{\text{wall}}^R}_{\text{wall penalty}} + \underbrace{\rho_{\text{move}} \mathbb{I}_{\text{move}}^R}_{\text{move reward}} + \underbrace{r_{\text{dist}}^R(\Delta d_t)}_{\text{distance shaping}}, \quad (4.6)$$

with $r_{\text{dist}}^R(\Delta d_t) = -r_{\text{dist}}^C(\Delta d_t)$. Due to asymmetric shaping (notably the Catcher-only step penalty), the shaped reward is not strictly zero-sum at intermediate steps, even though the terminal capture component is adversarial.

4.3.2 Step-Level Diagnostics (info Structure)

For analysis and reproducibility, each environment step produces a detailed diagnostic record. This includes:

- outcome flags: capture indicator, termination indicator, and current timestep,
- stochastic flags: action-failure indicators for both agents,
- transitions: old and new positions for Catcher and Runner,
- distances: d_t , d_{t+1} , and Δd_t ,
- movement diagnostics: wall-hit and moved indicators for both agents,
- full reward decomposition for both agents, and the Runner reward value.

This information is used by both training loops (when the Runner is also learning) and the logging layer.

4.4 Tabular State Encoding

Tabular learners require a single integer index for each joint-position state. We use the encoding

$$\text{sid}(s) = \text{id}_C \cdot N^2 + \text{id}_R, \quad \text{id}_C = x^C \cdot N + y^C, \quad \text{id}_R = x^R \cdot N + y^R, \quad (4.7)$$

which maps $s = (x^C, y^C, x^R, y^R)$ into $\{0, \dots, (N^2)^2 - 1\}$. This representation is used by joint-action tables such as $Q[\text{sid}, a_C, a_R]$.

4.5 Agents in the Base Chasing Scenario

This scenario includes two policy families: a belief-based one-step planner (Fictitious Play) and a tabular game-theoretic learner (Minimax-Q). Both operate in the simultaneous-move setting.

4.5.1 Fictitious Play Agent (FP)

The FP agent maintains, for each encoded state sid , an empirical estimate of the opponent's action distribution. Concretely, it stores action counts $\text{counts}[\text{sid}, a]$ initialized with a small prior, then updates the counts after observing the opponent action at each step.

At decision time, FP computes the normalized opponent distribution $\hat{\pi}_{\text{opp}}(\cdot | s)$ and evaluates each candidate self action by one-step lookahead. The lookahead explicitly marginalizes over:

- opponent actions sampled from $\hat{\pi}_{\text{opp}}(\cdot | s)$, and
- the four independent failure outcomes (fail / no-fail) for both agents, using p_{fail} .

For each candidate action, FP accumulates (i) the probability of immediate capture and (ii) the expected post-step Manhattan distance in non-capture outcomes. The Catcher selects actions that maximize capture probability and, secondarily, minimize expected distance; the Runner uses the opposite preference ordering (minimize capture probability and maximize distance). FP does not learn a value function; its adaptivity comes from updating the empirical opponent model.

4.5.2 Minimax-Q Agent

Minimax-Q is implemented with a joint-action Q-table $Q(\text{sid}, a_C, a_R)$ and a per-state maximin mixed strategy for the Catcher computed via linear programming (LP). At a given state s , the Catcher is treated as the row player and the Runner as the column player. The LP solves for a mixed policy $\pi_C(\cdot | s)$ that maximizes the guaranteed value $v(s)$ against any Runner action:

$$\max_{\pi_C, v} v \tag{4.8}$$

$$\text{s.t. } \sum_{a_C \in \mathcal{A}} \pi_C(a_C | s) Q(s, a_C, a_R) \geq v, \quad \forall a_R \in \mathcal{A}, \tag{4.9}$$

$$\sum_{a_C \in \mathcal{A}} \pi_C(a_C | s) = 1, \quad \pi_C(a_C | s) \geq 0. \tag{4.10}$$

The resulting π_C and $v(s)$ are cached per state and recomputed only when the local Q-values are updated.

Exploration schedule. Minimax-Q uses an ϵ -greedy exploration schedule with linear decay from $\epsilon_{\text{start}} = 0.20$ to $\epsilon_{\text{end}} = 0.05$ over 5000 episodes. With probability ϵ , the acting agent selects a uniformly random action; otherwise it follows the current policy.

Action selection. For the Catcher, the greedy action is sampled from the computed mixed strategy $\pi_C(\cdot | s)$. For the Runner, a best response is computed against the Catcher mixture by minimizing the expected Catcher payoff:

$$a_R^*(s) \in \arg \min_{a_R \in \mathcal{A}} \sum_{a_C \in \mathcal{A}} \pi_C(a_C | s) Q(s, a_C, a_R). \quad (4.11)$$

Update rule. Let $(s_t, a_t^C, a_t^R, r_t^C, s_{t+1})$ denote a transition with the Catcher reward returned by the environment. The Q-update is

$$Q(s_t, a_t^C, a_t^R) \leftarrow (1 - \alpha) Q(s_t, a_t^C, a_t^R) + \alpha (r_t^C + \gamma v(s_{t+1})), \quad (4.12)$$

with $\alpha = 0.10$ and $\gamma = 0.95$. If the transition is terminal, the target reduces to r_t^C .

4.6 Training, Evaluation, and Outputs (Base Chasing)

4.6.1 Matchups and Execution

Training is organized into matchup-specific loops, each producing training logs and periodic evaluation snapshots. The implementation runs the following matchups:

- FP vs FP,
- Minimax-Q self-play (one learner controls both sides: Catcher uses the maximin policy and Runner uses the best response),
- Minimax-Q vs FP,
- Minimax-Q vs Minimax-Q (two independent learners, one per role).

Experiments are executed for 10,000 episodes per matchup and repeated under different stochasticity levels, specifically $p_{\text{fail}} \in \{0.10, 0.20\}$.

4.6.2 Periodic Evaluation Protocol

Following the general protocol defined in Chapter 3, evaluation is performed every 50 training episodes. At each checkpoint, 10 evaluation episodes are run in a fresh environment instance with exploration disabled. The primary reported metrics are:

- **Capture rate:** fraction of evaluation episodes ending in capture,
- **Average steps-to-capture:** mean episode length over captured episodes,
- **Average return:** mean cumulative Catcher reward per episode.

4.6.3 Step-Level Logging (Excel Output)

The base scenario logs step-level transitions to two Excel workbooks:

- a training log workbook, and
- an evaluation log workbook.

Each row corresponds to one environment step and includes run metadata, episode/step indices, state and action fields (including action names), next-state fields, reward values for both agents, terminal flags, stochastic failure indicators, distance measures, and the full reward decomposition. Agent-specific fields (e.g. FP opponent distributions or Minimax-Q mixed strategies and value estimates) are also recorded when applicable. This structured logging supports the notebook-based post-processing used for the experiment analysis presented in Chapter 6.

Chapter 5

Wumpus Chase

5.1 Scenario Overview and Design Goals

The *Wumpus Chase* scenario extends the base pursuit–evasion game by embedding the chase mechanics within a Wumpus-style gridworld. In addition to adversarial interaction, agents must reason about *environmental hazards* (pits and a Wumpus), *local perceptual cues* (breeze and stench), and a *secondary terminal objective* (reaching a treasure cell). This setting intentionally introduces competing pressures: pure pursuit may be suboptimal if it drives an agent into hazard regions, while purely risk-averse navigation can allow the opponent to secure treasure or survive until timeout.

A central design decision is **fixed asymmetric roles** throughout all experiments:

- Agent A is always the *Chaser*.
- Agent B is always the *Runner*.

To avoid ambiguous outcomes (especially under simultaneous moves and multiple terminal triggers), the environment enforces a strict priority ordering of terminal conditions (Section 5.3.4). This yields a deterministic terminal label per episode and effectively eliminates draws at the environment level.

5.2 Map Layout System and World Elements

Each environment instance is defined by a *layout* specifying the static objects placed on the grid:

- **Obstacles:** impassable cells that block movement,
- **Pits:** lethal hazard cells,
- **Wumpus:** a lethal hazard cell,
- **Treasure:** a terminal goal cell.

From the hazard placements, the layout precomputes two perceptual regions:

- **Breeze cells:** cells adjacent (4-neighborhood) to any pit,

- **Stench cells:** cells adjacent (4-neighborhood) to the Wumpus.

We use multiple predefined layouts to vary difficulty and to test robustness. A compact 5×5 map supports fast iteration and sanity checks, while 7×7 maps increase complexity by expanding the state space and typically increasing hazard pressure and path-planning requirements. Across layouts, the treasure is placed such that reaching it is feasible but not trivial, forcing agents to trade off risk, distance-to-opponent, and distance-to-treasure.

5.3 Extended Environment Specification

5.3.1 Grid, Coordinates, and State

The world is an $N \times N$ grid (commonly $N \in \{5, 7\}$). The internal episode state tracks the timestep t and the agent positions (x_t^A, y_t^A) and (x_t^B, y_t^B) . Coordinates follow row/column conventions: x increases downward (row index behavior) and y increases to the right.

The environment supports two observation modes:

- **Fully observable (default):**

$$s_t = (x_t^A, y_t^A, x_t^B, y_t^B). \quad (5.1)$$

- **Partially observable:** a compact binary tuple encoding whether each agent is currently in breeze/stench regions,

$$o_t = (\mathbb{I}_{\text{breeze}}^A, \mathbb{I}_{\text{stench}}^A, \mathbb{I}_{\text{breeze}}^B, \mathbb{I}_{\text{stench}}^B).$$

Unless stated otherwise, experiments use the fully observable configuration to align tabular state encodings with joint positions and to focus comparisons on learning dynamics rather than partial observability.

5.3.2 Action Space and Stochastic Dynamics

Both agents act simultaneously using the same action set

$$\mathcal{A} = \{\text{STAY, UP, DOWN, LEFT, RIGHT}\}.$$

Stochastic actuation is introduced as in the base scenario: each agent fails to execute its intended action independently with probability p_{fail} (default 0.10). On failure, the agent remains in place; importantly, this failure event is treated separately from collisions with walls/obstacles (i.e., an actuation failure does not automatically imply a bump).

If an intended movement would cross the grid boundary or enter an obstacle cell, the agent remains in place and a **bump** indicator is set. This signal is logged step-by-step and also used for penalties in the reward function (Section 5.5.1).

5.3.3 Reset and Valid Start States

At reset, the timestep is initialized to $t = 0$ and both agents are sampled uniformly from the set of *free cells*. Free cells exclude obstacles, pits, the Wumpus, and the treasure, and the two starting positions are forced to be distinct. This avoids trivial initializations (immediate hazard deaths or instant treasure acquisition) and ensures that early episode dynamics reflect decision-making rather than deterministic terminal events.

5.3.4 Terminal Outcome Priority (No-Draw Runtime)

A key design goal is to remove outcome ambiguity by enforcing a strict priority ordering of terminal conditions. At each step, termination is resolved by the following sequence:

1. **Capture:** if the agents land on the same cell or swap positions in one step, the outcome is `A_WIN`.
2. **Treasure:** if one agent reaches the treasure first, that agent wins; if both reach the treasure on the same step, the tie-break favors Agent A (`A_WIN`).
3. **Hazards:** if Agent B dies (pit or Wumpus), then `A_WIN`; else if Agent A dies, then `B_WIN`. (If both die on the same step, the rule still yields `A_WIN` due to the “B dies” clause.)
4. **Timeout:** if $t \geq t_{\max}$ with no earlier terminal event, the outcome is `B_WIN`.

This rule eliminates draws at the environment level. For compatibility with analysis pipelines, the evaluation interface may still expose a draw-rate field, which should remain near zero under this terminal logic.

5.4 Perception Model

Each agent is associated with a Wumpus-style percept tuple:

`(breeze, stench, glitter, bump, scream),`

where `breeze` and `stench` indicate adjacency to pits and the Wumpus respectively, `glitter` indicates being on the treasure cell, and `bump` indicates an invalid move attempt (wall/obstacle). The `scream` field is reserved for future extensions and is always `False` in the current version. Percepts are logged step-by-step and are also incorporated into reward shaping (via penalties for occupying breeze/stench regions).

5.5 Reward Design and Decomposition

5.5.1 Reward Parameters and Interface Convention

The environment computes a decomposed reward for *both* agents at every step:

- `reward_a` for the Chaser (Agent A),
- `reward_b` for the Runner (Agent B).

However, for compatibility with standard single-reward interfaces, the step function returns only the Chaser reward directly, while the Runner reward is exposed inside the diagnostic `info` structure (Section 5.5.3). This convention is important for training: whenever Agent B is a learning agent, its update uses the `reward_b` value retrieved from `info`.

Default reward hyperparameters are:

`outcome_reward = 10.0, step_penalty = 0.01, pit_penalty = 10.0, wumpus_penalty = 10.0, obstacle_`

5.5.2 Outcome, Hazard, and Shaping Terms

Rewards are constructed as the sum of interpretable components:

- **Outcome term:** $\pm \text{outcome_reward}$ at termination depending on the winner.
- **Step penalty:** applied only to Agent A during non-terminal steps to encourage faster resolution.
- **Hazard penalties:** large negative penalties applied to whichever agent steps into a pit or the Wumpus.
- **Obstacle/bump penalty:** penalizes invalid moves against walls or obstacles.
- **Perception penalties:** penalize occupying breeze/stench regions (risk-sensitive shaping).
- **Chase-distance shaping:** encourages Agent A to reduce distance to Agent B, and Agent B to increase it.
- **Treasure-distance shaping:** rewards progress toward the treasure for each agent.

Let d_t^{AB} be the Manhattan distance between agents, and d_t^{iT} the Manhattan distance from agent $i \in \{A, B\}$ to the treasure. The chase shaping depends on $\Delta d_t^{AB} = d_{t+1}^{AB} - d_t^{AB}$, while treasure shaping depends on $\Delta d_t^{iT} = d_{t+1}^{iT} - d_t^{iT}$.

At a high level, the total reward for Agent A takes the form

$$r_t^A = r_{\text{outcome}}^A + r_{\text{step}}^A + r_{\text{hazard}}^A + r_{\text{bump}}^A + r_{\text{percept}}^A + r_{\text{chase}}^A + r_{\text{treasure}}^A, \quad (5.2)$$

and similarly for Agent B. The chase shaping is sign-inverted between roles: Agent A receives positive shaping when d^{AB} decreases, while Agent B receives positive shaping when d^{AB} increases. Treasure shaping is computed separately for each agent relative to its own distance-to-treasure change.

5.5.3 Step-Level Diagnostics (info Structure)

Each step exposes a rich diagnostic record that includes:

- terminal metadata (outcome label, capture flag, winner role, timestep),
- transition metadata (old/new coordinates, action names, actuation failure and bump flags),
- hazard flags (pit contact, Wumpus contact, death indicators),
- perceptual flags (breeze/stench/glitter),
- distances before/after (agent distance and treasure distance),
- full reward decomposition for both agents, including the Runner reward value.

This **info** structure is the primary bridge between environment mechanics, learning updates (especially when Agent B learns), and the logging subsystem used for the evaluation analysis in Chapter 6.

5.6 Tabular State Encoding and Game-Theoretic Utilities

5.6.1 Tabular State Encoding

For tabular methods under full observability, we use a joint-position encoding scheme that maps (x^A, y^A, x^B, y^B) to a single integer state id:

$$\text{sid}(s) = \text{id}_A \cdot N^2 + \text{id}_B, \quad \text{id}_A = x^A \cdot N + y^A, \quad \text{id}_B = x^B \cdot N + y^B. \quad (5.3)$$

This yields a flat index over $(N^2)^2$ joint states and supports joint-action Q-tables of the form $Q[\text{sid}, a_A, a_B]$. The same encoding is used consistently across Minimax-Q, Nash-Q, and Dyna-style tabular agents to ensure comparable state representations.

5.6.2 Maximin Linear Program (LP) for Security Policies

Minimax-Q requires computing a security (maximin) mixed strategy for the row player given a state-local payoff matrix. We solve this by a standard linear program that maximizes a lower bound game value v subject to simplex constraints on the mixed strategy and security constraints against all opponent actions. If the LP fails or is numerically unstable for a given state (e.g., due to degenerate matrices), we fall back to a uniform mixture and estimate the value under that policy. This utility is used directly by Minimax-Q and also supports robust action-selection routines that require adversarially safe mixtures.

5.7 Learning Agents Supported in Wumpus Chase

Wumpus Chase supports a broader set of learning methods than the base scenario. All agents operate under the fixed-role semantics ($A = \text{Chaser}$, $B = \text{Runner}$), and each learning update uses the reward signal appropriate to its role (Agent A uses the direct environment reward; Agent B uses the `reward_b` value available in `info`).

5.7.1 Fictitious Play (FP)

FP is implemented as a belief-based agent that maintains per-state opponent action counts and evaluates candidate actions under the estimated opponent distribution and the stochastic failure model. In Wumpus Chase, FP's scoring is an approximation to the full environment reward: it captures key outcome structure (capture/terminal events) and movement/shaping signals (obstacles and distance terms), but it does not explicitly reproduce every hazard/perception penalty. FP therefore serves primarily as a competitive, non-tabular baseline that can be paired against learning agents to stress-test robustness.

5.7.2 Minimax-Q

Minimax-Q is a tabular joint-action learner designed for adversarial (zero-sum) interaction. It maintains $Q(s, a_A, a_B)$ and computes state-wise security policies by solving the maximin LP (Section 5.6.2). Exploration follows a linear ϵ -decay schedule (default $\epsilon : 0.20 \rightarrow 0.05$ over 5000 episodes). In cross-play, the algorithm can be assigned to either role; when controlling Agent B, the same security-policy computation is applied to the transposed payoff structure to obtain a robust runner policy.

5.7.3 Nash-Q

Nash-Q targets general-sum stochastic games and maintains separate joint-action Q-tables, $Q_A(s, a_A, a_B)$ and $Q_B(s, a_A, a_B)$. At each state, a Nash equilibrium (or a stable approximation) is computed to obtain mixed strategies and state values that define TD targets. Since exact equilibrium solvers can be expensive or fail on some payoff matrices, the implementation supports multiple equilibrium-computation routines with practical fallbacks (ensuring training continues even when an exact solver is unavailable). Exploration typically uses a higher initial rate than Minimax-Q (default $\epsilon : 0.30 \rightarrow 0.05$ over 8000 episodes).

5.7.4 Dyna-Q and Dyna-Q+

Dyna-Q integrates direct Q-learning with model-based planning. The agent learns a transition model online from real interactions and, after each real step, performs n_{planning} additional simulated updates by sampling transitions from the learned model (default $n_{\text{planning}} = 10$). Dyna-Q+ extends this by adding an exploration bonus *during planning updates only* (not during real interaction), encouraging revisiting state-action pairs that have not been tried recently. This is particularly relevant in Wumpus Chase, where hazards and treasure regions create localized risk/reward structures and the agent benefits from systematically exploring alternative corridors without repeatedly incurring real hazard costs.

5.7.5 Deep Q-Networks (Optional DQN Stack)

An optional DQN stack is included for function approximation beyond tabular settings. Supported variants include standard DQN, Double DQN, Dueling DQN, and combined Dueling Double DQN. The feature representation includes normalized positional features and distance-based descriptors (agent distance, distances to treasure and hazards), optionally augmented with cue indicators. Training uses experience replay and a target network updated at a fixed frequency. These deep-RL components are evaluated separately as experimental baselines and are not required for the core tabular results.

5.8 Training, Evaluation, and Logging Protocol

5.8.1 Experimental Matchups (What We Trained)

Although the implementation supports many pairwise combinations, we **do not** run an exhaustive “all-vs-all” tournament. Instead, experiments are defined as a curated set of canonical matchups that cover: (i) self-play baselines within each method family, and (ii) selected cross-family pairings that stress different aspects of adversarial/hazard-aware behavior.

We organize matchups using the following naming convention: **X_vs_Y** indicates Agent A (Chaser) uses method X and Agent B (Runner) uses method Y, while **X_selfplay** indicates both roles use X. The canonical matchup inventory is:

- **Self-play baselines:** `fp_selfplay`, `minimaxq_selfplay`, `nashq_selfplay`, `dynaq_selfplay`, `dynaq_plus_selfplay`.

- **Cross-play (selected pairings):** `fp_vs_minimaxq`, `fp_vs_nashq`, `fp_vs_dynaq`, `minimaxq_vs_fp`, `minimaxq_vs_nashq`, `minimaxq_vs_dynaq`, `nashq_vs_minimaxq`, `nashq_vs_dynaq`, `dynaq_vs_fp`, `dynaq_vs_minimaxq`, `dynaq_vs_nashq`, and the Dyna-Q+ extensions `dynaq_plus_vs_fp`, `dynaq_plus_vs_minimaxq`, `dynaq_plus_vs_nashq`, `dynaq_plus_vs_dynaq`.
- **Optional deep-RL baseline:** `dqn_selfplay` (reported separately when enabled).

Chapter 6 reports results only for the matchups that were actually executed and logged in the final experimental runs.

5.8.2 Training Loop and Update Semantics

Training is episodic. In each episode, both agents select actions simultaneously, the environment transitions to s_{t+1} , and termination is resolved via the priority ordering in Section 5.3.4. The learning signal is role-aware and uses the decomposed reward interface:

- Agent A learners update from the direct step reward r_t^A .
- Agent B learners update from r_t^B retrieved from `info`.

This separation is essential in cross-play, where the learner may control either role.

Across tabular learners, we use consistent base hyperparameters unless a method requires otherwise:

$$\gamma = 0.95, \quad \alpha = 0.10,$$

with ϵ -greedy exploration using method-specific decay schedules (Minimax-Q typically starts lower than Nash-Q; Section 5.7). For Dyna methods, each real transition triggers additional planning updates (default $n_{\text{planning}} = 10$), and Dyna-Q+ applies a novelty bonus during those simulated updates only.

Episode budgets are chosen to reflect method complexity: shorter baseline runs are used for quick validation, while cross-family matchups and Dyna/Nash variants typically require longer training to stabilize. In our default configuration, evaluation checkpoints are scheduled at fixed episode intervals (typically every 1000 episodes). For shorter baseline runs (e.g., 500-episode sanity checks), we report final evaluation only.

5.8.3 Evaluation Protocol and Metrics

Evaluation follows the shared protocol from Chapter 3, with the important Wumpus-specific note that **draws should not occur** under the terminal logic (Section 5.3.4). During evaluation we report:

- **Win rate (Agent A / Chaser):** fraction of episodes with outcome `A_WIN`,
- **Runner win rate (Agent B):** fraction of episodes with outcome `B_WIN`,
- **Average steps:** mean episode length (including early terminations),
- **Average return:** mean cumulative reward of Agent A (with reward decomposition retained for diagnosis).

Because Wumpus Chase has multiple terminal mechanisms (capture, treasure, hazards, timeout), Chapter 6 also analyzes outcome composition (e.g., how often B wins by reaching treasure versus A winning by hazard) and links these outcomes to behavioral diagnostics such as distance dynamics and hazard exposure.

5.8.4 Step-Level Logging for Post-Hoc Analysis

All experiments are logged at **step granularity** (one row per environment step). This design is intentional: Wumpus Chase behaviors are best explained by time-resolved signals (hazard contacts, percept flags, bump/failure events, distance-to-opponent and distance-to-treasure trajectories), not only by episode-level aggregates.

Each logged row includes:

- run context (matchup label, phase, roles),
- environment configuration snapshot (grid size, p_{fail} , t_{max} , reward hyperparameters),
- layout metadata (serialized obstacle/pit sets and the locations of Wumpus and treasure),
- transitions (old/new coordinates, actions, actuation failure, bump),
- terminal/hazard indicators (capture, treasure, pit/Wumpus contact, death, winner),
- distance diagnostics (A–B distance and distances to treasure before/after),
- reward decomposition for both agents (including `reward_b`).

This logging layer directly supports the notebook-based evaluation analysis presented in Chapter 6, where we compute rolling performance curves, outcome breakdowns, spatial visitation heatmaps, distance-change mixtures, and reward-component attributions.

5.9 Extension: Opponent Diversity and Robustness

5.9.1 Research Question and Experimental Scope

The extension investigates whether training against a diverse population of opponents improves generalization to unfamiliar strategies, and whether varying environmental stochasticity during training improves performance when action failures become more frequent. It separates three experimental factors: opponent diversity, randomization of the action-failure probability, and adaptive allocation of training episodes. The learning algorithm is held constant across conditions, allowing comparisons to focus on the training distribution.

This is a supplementary, controlled notebook experiment with its own environment configuration and reward specification. The reported pilot uses the compact 5×5 layout, a horizon of 30 simultaneous steps, and separate learners for the Chaser and Runner. Obstacles occupy (1, 1) and (2, 1), the pit is at (2, 2), the Wumpus at (3, 3), and treasure at (1, 4). Starting positions are distinct safe cells, excluding treasure. Each agent’s selected action independently fails with probability p_{fail} . Terminal resolution follows capture, treasure, hazards, and timeout in that order; capture includes agents exchanging positions. Timeout awards victory to the Runner. These role-dependent objectives motivate reporting results separately for the two roles.

5.9.2 State Representation, Rewards, and Common Learner

The extension observes both agent positions, remaining time, and the current p_{fail} . A 16-dimensional feature vector contains normalized coordinates, inter-agent distance, distances from both agents to treasure and the Wumpus, breeze and stench indicators, normalized remaining time, the failure probability, and the learner’s role. Consequently, the experiment measures robustness under *observed* changes in stochasticity; inferring an unknown failure probability is outside its scope.

Rewards in this controlled study are strictly zero-sum. Let b_A be +10 for a Chaser victory, −10 for a Runner victory, and zero on nonterminal transitions. With Manhattan distance d , treasure position T , and grid width N , the shaping potential is

$$\Phi_A(s) = \frac{-0.5d(a, b) - 0.25d(a, T) + 0.25d(b, T)}{2(N - 1)}.$$

The transition rewards are

$$r_A(s, s') = b_A + \gamma\Phi_A(s') - \Phi_A(s), \quad r_B(s, s') = -r_A(s, s'),$$

with $\Phi_A(s') = 0$ at every terminal outcome, including timeout. This provides a distance-based learning signal while retaining an explicitly competitive reward structure.

Every experimental condition uses a Dueling Double DQN learner with two hidden layers of 64 units, experience replay, and a target network. The dueling architecture represents action values through a state-value stream and centered action advantages. Double DQN selects the next action with the online network and evaluates it with the target network. The pilot uses $\gamma = 0.95$, Adam with learning rate 5×10^{-4} , batches of 64, replay capacity 50,000, and a warm-up of 256 transitions. Learning updates occur every four environment steps and target synchronization every 200 steps. Exploration decreases linearly from $\epsilon = 1$ to 0.05 over the first 70% of the training budget and then remains at 0.05.

5.9.3 Opponent Banks and Training Conditions

Opponent banks contain frozen policies from four families: Fictitious Play, Minimax-Q, a Dyna-style Minimax learner with model-based planning, and DQN. Bank policies receive 5,000 bootstrap steps against alternating random and safe scripted sparring opponents. The training pool includes final snapshots from all four families and an additional intermediate DQN checkpoint. Sampling first selects a family and then a policy within that family, so the extra DQN snapshot does not increase its family’s total weight.

Training, diagnostic validation, and test banks use separate seed assignments. Independent test-bank policies assess transfer to new instances of familiar algorithm families. Two additional scripted families, *treasure* and *ambush*, are reserved exclusively for testing unfamiliar strategies. The pilot includes all four learned families during population training; a leave-one-learned-family-out experiment is a possible further evaluation.

Table 5.1 summarizes the seven executed conditions. Conditions A and C each include separate fixed-FP and fixed-Minimax controls. The corresponding A/C pair uses the same fixed opponent snapshot. For randomized conditions, the failure probability is sampled at the start of each episode and remains constant throughout that episode.

In condition E, a cell i denotes an opponent-family/failure-probability combination. Difficulty starts at $d_i = 0.5$ and is recalibrated at 25%, 50%, and 75% of the training budget

Condition	Training opponents	Training p_{fail}
A-FP	One fixed FP policy	0.10
A-MM	One fixed Minimax policy	0.10
B	Uniform family sampling	0.10
C-FP	One fixed FP policy	$\{0.05, 0.10, 0.15\}$
C-MM	One fixed Minimax policy	$\{0.05, 0.10, 0.15\}$
D	Uniform family sampling	$\{0.05, 0.10, 0.15\}$
E	Adaptive family-noise sampling	$\{0.05, 0.10, 0.15\}$

Table 5.1: Training conditions for the opponent-diversity extension. Conditions C and D sample noise levels uniformly; E adapts their joint distribution with opponent families.

using eight evaluation episodes per cell against training-pool opponents, with independent validation random streams. If $\hat{w}_i$ is the measured win rate, the update and sampling probability are

$$d_i \leftarrow 0.5d_i + 0.5(1 - \hat{w}_i), \quad q_i = \frac{0.2}{K} + 0.8 \frac{(d_i + 0.05)^2}{\sum_{j=1}^K (d_j + 0.05)^2},$$

where K is the number of cells. This emphasizes difficult combinations while retaining a minimum probability for every cell. Test results never influence this schedule. The separate validation bank supports opponent-quality diagnostics and is not the source of these adaptive calibration opponents.

5.9.4 Evaluation Protocol and Experimental Budget

Each learner receives 20,000 real training steps. Separate runs cover both roles and each seed in $\{11, 22, 33\}$. The resulting 42 learners consume 840,000 training steps. This equalizes real learner experience; opponent-bank preparation and adaptive calibration require additional interaction and computation.

Evaluation freezes both participants and disables exploratory actions. Stochastic policies can still sample their prescribed mixed strategies. Each learner faces ten opponents: four final training-bank snapshots, four independently trained test-bank snapshots, and the two withheld scripted strategies. Training-bank membership does not imply that a fixed-opponent learner encountered every such policy during training. Each pairing is evaluated for 100 episodes at

$$p_{\text{fail}} \in \{0, 0.05, 0.10, 0.15, 0.20, 0.30\},$$

yielding 252,000 evaluation episodes overall. Separate random streams control environmental failures and the two participants' actions. Shared scenario seeds across conditions support paired comparisons.

The primary endpoint is the macro-averaged win rate against the two withheld scripted families at $p_{\text{fail}} \in \{0.20, 0.30\}$, beyond the randomized training range. Episode outcomes are averaged within each policy/noise combination, followed by equal weighting of families and noise levels. Additional diagnostics include returns, episode duration, hazard exposure, and terminal-outcome composition. Uncertainty is summarized with nominal 95% Student- t confidence intervals across the three training seeds. Evaluation episodes estimate each

trained policy’s performance; the training seeds provide the independent learner replications. Paired contrasts first average the two fixed controls within each seed. The reported intervals are unadjusted for multiple comparisons.

Results and role-dependent interpretations are reported in Section 6.3.

5.9.5 Limitations and Further Experiments

The pilot offers evidence that opponent selection and environmental randomization can affect generalization, with benefits depending on the role and baseline. Its scope is limited to one fixed map, three training seeds, two withheld scripted families, and a modest learner budget. The number and strength of opponent snapshots also constrain the diversity represented by the population. Observing the failure probability allows policies to condition on noise and does not establish robustness when that quantity is hidden.

Further experiments can increase training seeds and budgets, evaluate independent layouts, expand opponent banks with stronger and behaviorally distinct policies, and withhold entire learned families during training. Comparing observed and hidden failure probabilities would separate noise-conditioned generalization from adaptation to unknown dynamics. Evaluations under matched total interaction or computation budgets would also clarify the cost of adaptive sampling. These extensions would help determine when population training provides reliable gains and when a suitable fixed opponent is sufficient.

Chapter 6

Experiments Results

6.1 Evaluation Results: Chase

This section reports evaluation results for the base Chase scenario using step-level evaluation logs aggregated into rolling metrics and episode-level summaries. We focus on (i) rolling capture rate and rolling steps, (ii) steps-to-capture distributions, (iii) controlling for initial difficulty via opening distances, (iv) empirical move-failure rates under p_{fail} , and (v) qualitative spatial behavior via visitation and capture-location heatmaps. Unless stated otherwise, evaluation episodes terminate either on capture or at the maximum episode length $t_{\text{max}} = 30$.

6.1.1 Rolling performance across evaluation episodes

Figures 6.1–6.2 summarize evaluation performance under two environment stochasticity settings ($p_{\text{fail}} \in \{0.1, 0.2\}$). We plot (left) the rolling capture rate and (right) the rolling number of steps per episode.

Across the matchups where captures occur regularly (e.g., `fp_vs_fp`, `minimaxq_selfplay`, `minimaxq_vs_minimaxq`), we observe two consistent patterns: (i) the rolling capture rate rises rapidly from early episodes and stabilizes at a high plateau, and (ii) the rolling steps decrease during early evaluation before stabilizing in the mid-teens. Among these, `minimaxq_vs_minimaxq` shows the strongest overall profile, combining a high stable capture-rate plateau with the lowest stable steps-to-capture.

In contrast, the configuration labeled `minimaxq_vs_fp` exhibits near-zero rolling capture rate and a rolling step count pinned close to the time limit, indicating that episodes predominantly terminate by timeout rather than capture in this specific pairing. This behavior is also consistent with the near-empty steps-to-capture histograms for this run (Section 6.1.2).

Finally, increasing the action failure probability from $p_{\text{fail}} = 0.1$ to 0.2 does not change the qualitative ordering across matchups: the strongest matchups remain strong, while the low-capture pairing remains low-capture. This indicates that the observed ranking is not an artifact of a particular stochasticity setting.

6.1.2 Steps-to-capture distributions

Figure 6.3 shows histograms of steps-to-capture for the evaluation episodes where a capture occurs (faceted by run and p_{fail}). For the configurations with high capture rates, the dis-

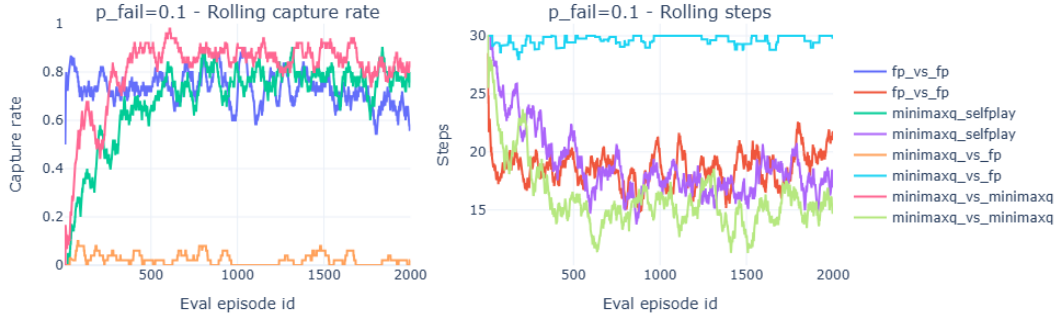


Figure 6.1: Chase evaluation (rolling metrics) under $p_{\text{fail}} = 0.1$: rolling capture rate (left) and rolling steps (right) for all run configurations.

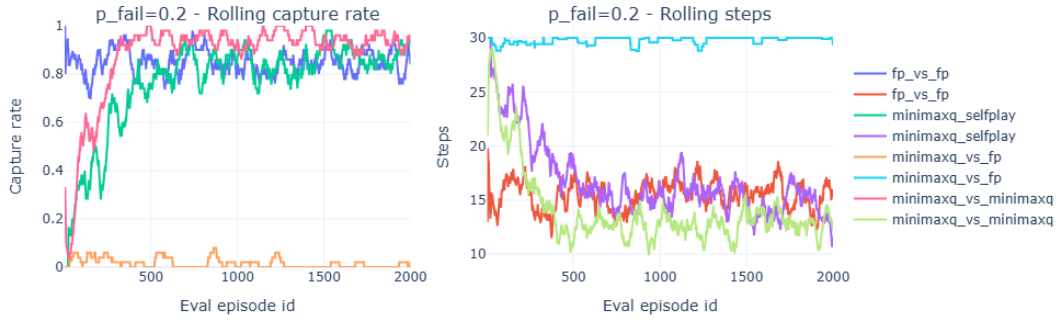


Figure 6.2: Chase evaluation (rolling metrics) under $p_{\text{fail}} = 0.2$: rolling capture rate (left) and rolling steps (right) for all run configurations.

tributions concentrate at relatively low step counts (typically below 20 steps), with a right tail toward the maximum episode length. This tail reflects more difficult episodes (e.g., unfavorable initial geometry or compounding action failures) where capture is achieved late.

The `minimaxq_vs_minimaxq` pairing exhibits the most favorable distributional profile: more probability mass at lower steps and fewer episodes accumulating near the time limit compared to other capture-heavy runs. Conversely, the `minimaxq_vs_fp` column contains very few captured episodes, consistent with timeout-dominated evaluation for that matchup.

6.1.3 Controlling for initial difficulty: opening Manhattan distance

To ensure that performance differences are not driven by systematically easier initial placements, we track the rolling opening Manhattan distance (distance at step 1) across evaluation episodes (Figure 6.4). Across all runs and both failure probabilities, the opening distance stays within a narrow band with only minor fluctuations. This indicates that eval-



Figure 6.3: Histogram of steps-to-capture (captured episodes only), faceted by run configuration (columns) and failure probability p_{fail} (rows).

uation episodes are comparable in initial difficulty across matchups, and that differences in capture outcomes and steps are primarily attributable to policy behavior rather than initialization bias.



Figure 6.4: Rolling opening Manhattan distance (step 1) across evaluation episodes, stratified by run and p_{fail} .

6.1.4 Empirical move-failure rate under p_{fail}

Figure 6.5 reports the observed rolling move-failure rate for the catcher and runner, stratified by run and by the configured failure probability. The primary qualitative check is that the curves clearly separate between the two settings: the $p_{\text{fail}} = 0.2$ row exhibits higher failure rates than the $p_{\text{fail}} = 0.1$ row across all matchups, confirming that the environment-level stochasticity is reflected in the logs.

A second consistent pattern is that the runner’s observed failure rate is typically higher than the catcher’s in most matchups. This suggests that the measured “failure” event is not purely a constant exogenous coin-flip but is also influenced by the interaction between policy and environment constraints (e.g., attempting moves that do not result in displacement, or states where intended actions are more often ineffective). In other words, beyond the configured p_{fail} , agent behavior can modulate the empirical rate at which intended moves fail to translate into executed displacement.

Notably, the mismatch `minimaxq_vs_fp` (the timeout-dominated pairing identified in Section 6.1.1) does not exhibit an anomalous failure-rate signature that would trivially explain the lack of captures; rather, its failure curves remain in the same general range as other runs under the same p_{fail} . This supports the interpretation that the low capture rate is driven primarily by strategic interaction (policy mismatch) rather than by an unusually high mechanical failure rate.

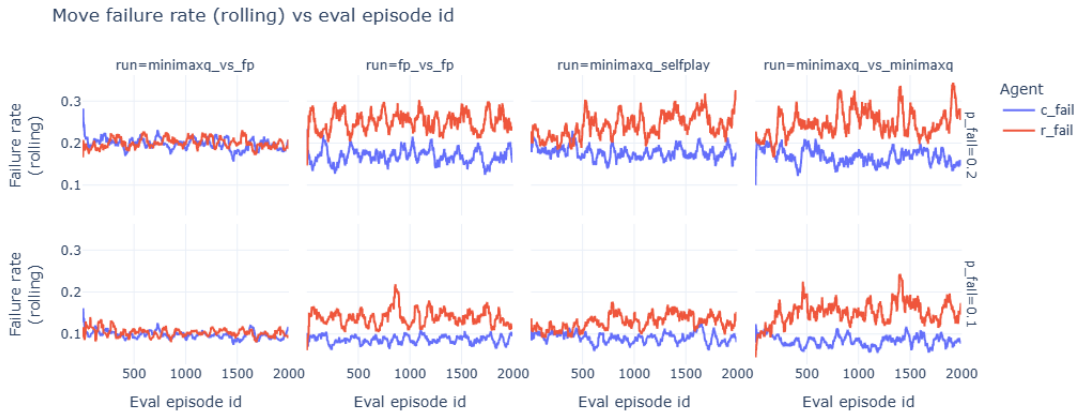


Figure 6.5: Rolling move-failure rate versus evaluation episode id for catcher (`c_fail`) and runner (`r_fail`), faceted by run (columns) and failure probability p_{fail} (rows).

6.1.5 Qualitative spatial behavior: visitation and capture locations

Figure 6.6 provides qualitative insight into spatial behavior for `minimaxq_vs_minimaxq` at $p_{\text{fail}} = 0.1$, showing (i) catcher visitation density, (ii) runner visitation density, and (iii) capture locations (runner position post-move at capture).

Two patterns stand out. First, the catcher concentrates visitation mass in central cells, consistent with a “control-the-center” pursuit strategy that reduces the runner’s maneuvering options. Second, the runner exhibits relatively higher visitation along the perimeter (especially near edges/corners), consistent with a boundary-following evasive strategy. Capture locations are distributed across the grid with mild concentrations aligned with the runner’s preferred regions; this supports the interpretation that captures often occur when the pursuer intercepts or corners the runner near boundaries rather than exclusively in the center.

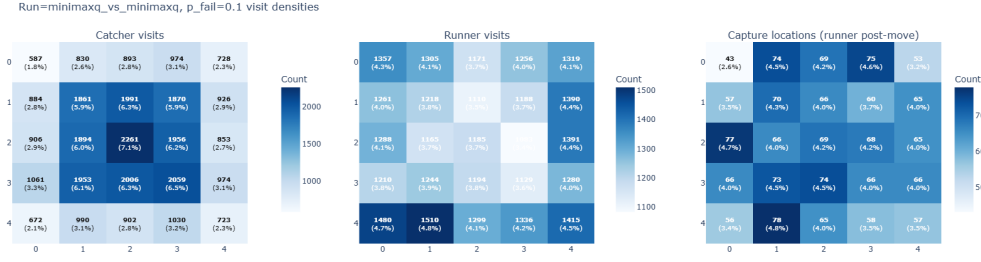


Figure 6.6: Spatial diagnostics for `minimaxq_vs_minimaxq` at $p_{\text{fail}} = 0.1$: catcher visitation (left), runner visitation (center), and capture locations (right).

6.1.6 Additional evaluation diagnostics in the notebooks (not included here)

The evaluation notebooks include further per-run diagnostics that we keep outside the main report to avoid figure overload. These are useful for “why” explanations when a specific pairing behaves atypically:

- **Wall-hit and movement outcome rates:** how frequently each agent hits walls, stays in place, or fails moves under different p_{fail} .
- **Distance-change mix:** per-step fractions of “closer / same / farther” transitions to quantify pursuit pressure and evasion effectiveness.
- **Reward shaping breakdown:** decomposition of the per-step reward into shaping components to identify what dominates learning signals in each matchup.
- **Trajectory visualizations (per seed/episode):** qualitative rollouts for selected runs to inspect behavior in representative episodes.

6.2 Evaluation Results: Wumpus Chase

This section reports evaluation results for the Wumpus Chase scenario. In contrast to the base Chase environment, episodes can terminate for multiple reasons (capture, treasure, hazards, timeout). For that reason, we present results in three layers: (i) aggregate success by role, (ii) terminal-outcome composition (how wins/losses are realized), and (iii) behavioral diagnostics that explain the aggregate patterns.

6.2.1 Overall performance by role

Figure 6.7 summarizes win-rate by agent and role. As a *Runner*, DynaQ+ achieves the highest win-rate (0.707), followed by DynaQ (0.654) and DQN (0.652). As a *Chaser*, DynaQ is the strongest performer (0.594), while DQN is substantially weaker in this role (0.348). Overall, the ranking highlights a clear role effect: planning-based methods dominate as Chasers, while the strongest Runner performance is achieved by DynaQ+.

Win-rate alone does not capture *how* the agent wins or loses. Figures 6.8 and 6.9 provide two complementary aggregates: average terminal reward and average terminal steps. The

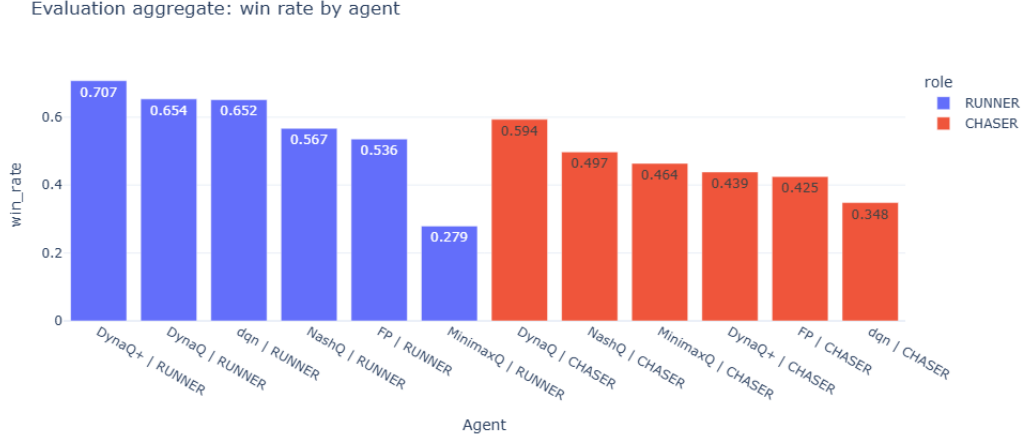


Figure 6.7: Evaluation aggregate: win rate by agent and role (Chaser vs Runner).

Runner role obtains strongly positive terminal rewards under DynaQ+ (4.104), with DQN Runner (3.013) and DynaQ Runner (2.992) also positive. For the Chaser role, DynaQ is the only method with a clearly positive average terminal reward (1.814), while the remaining Chasers are negative on this metric, and DQN Chaser is particularly low (-3.158). These reward aggregates align with the win-rate ranking and indicate that DynaQ achieves not only more wins as Chaser, but also higher-quality terminal outcomes.

Average terminal steps reveal a qualitative difference in episode dynamics. DynaQ+ Runner produces the longest episodes on average (27.46 steps), consistent with strategies that frequently survive to late termination. In contrast, DynaQ as Chaser terminates earlier on average (11.34 steps), indicating that its wins tend to be resolved more efficiently.



Figure 6.8: Evaluation aggregate: average terminal reward by agent and role.



Figure 6.9: Evaluation aggregate: average terminal steps by agent and role.

6.2.2 Terminal outcomes: what ends the episode

To interpret Wumpus Chase performance, it is essential to examine the distribution of terminal reasons. Figure 6.10 reports the terminal-reason mix by agent-role. Two dominant mechanisms appear across methods:

- **Runner wins are often timeout-driven.** In particular, DynaQ+ as Runner exhibits a high `timeout` share, which directly explains its longer episodes (Figure 6.9) and its strong Runner win-rate (Figure 6.7).
- **Chaser success frequently routes through treasure.** For DynaQ as Chaser, `chaser_treasure` is a major contributor to terminal outcomes. This indicates that effective Chaser policies in Wumpus Chase are not purely “capture-seeking”; they also exploit the secondary objective to close episodes decisively under the environment’s terminal priority rules.

Figure 6.11 shows the distribution of terminal steps conditioned on terminal reason. Timeouts concentrate near the evaluation horizon ($t_{\max} \approx 50$), whereas capture and treasure outcomes occur much earlier, with most mass concentrated in the first few steps. This separation clarifies how different “success styles” translate into episode-length statistics: timeout-heavy strategies inflate mean steps, while capture/treasure-heavy strategies resolve episodes earlier.

Finally, among hazard endings only, Figure 6.12 separates pit versus Wumpus as the hazard cause. Pits dominate hazard endings for both sides, indicating that (under the tested layouts) pit management is the primary hazard-avoidance challenge and a key driver of robust play.

6.2.3 Behavioral diagnostics

Action selection tendencies

Figure 6.13 reports action-share distributions by role and by run. Across runs, `STAY` is the largest single mass for both roles, and the remaining probability is split across cardinal

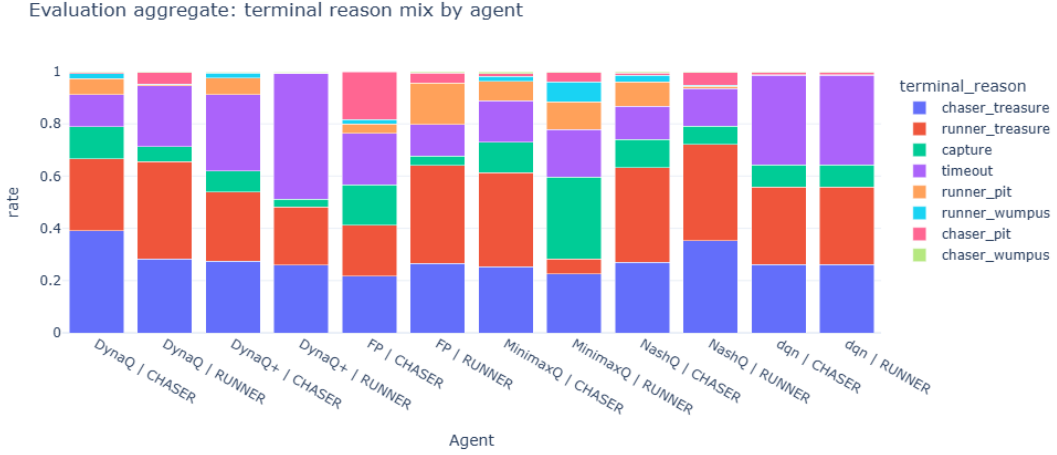


Figure 6.10: Evaluation aggregate: terminal reason mix by agent-role (stacked rates).

moves. This is consistent with the Wumpus setting: stationary actions can be rational when agents are managing hazard exposure, avoiding bump penalties, or preserving a favorable local configuration. At the same time, the pronounced **STAY** share provides a useful diagnostic when interpreted together with terminal reasons: runs with large timeout shares (Section 6.2.2) naturally coincide with policies that frequently avoid decisive repositioning.

Spatial control under pressure

To move beyond scalar aggregates, Figure 6.14 visualizes Runner position densities under two regimes for `dynaq_plus_selfplay` with $p_{\text{fail}} = 0.1$: “under pressure” ($d_{AB} \leq 2$) versus “outside pressure” ($d_{AB} > 2$). The two heatmaps differ substantially, indicating a regime-dependent policy. When pressured, the Runner concentrates in a subset of cells that are consistent with local evasive maneuvering; outside pressure, the distribution shifts, consistent with broader navigation behavior that can support long-horizon survival and favorable terminal outcomes (notably timeouts).

Distance-based progress signals

Figures 6.15 and 6.16 summarize two distance-based diagnostics: (i) favorable distance progress rate and (ii) mean goal distance. The favorable-progress metric highlights which agents consistently make locally beneficial distance changes. On this metric, **DynaQ** and **NashQ** are strongest for the Chaser role (both around 0.26), while **FP** is strongest for the Runner role (0.341).

Mean goal distance is informative but must be interpreted carefully. **DQN** attains the lowest mean goal distance for both roles, yet it does not achieve the strongest Chaser win-rate and it scores relatively low on favorable progress. This indicates that being “close to the goal on average” is not sufficient in Wumpus Chase: terminal outcomes depend on interaction effects (pressure, hazards, and the terminal priority rules), so goal proximity must be considered together with terminal-reason composition and risk exposure.

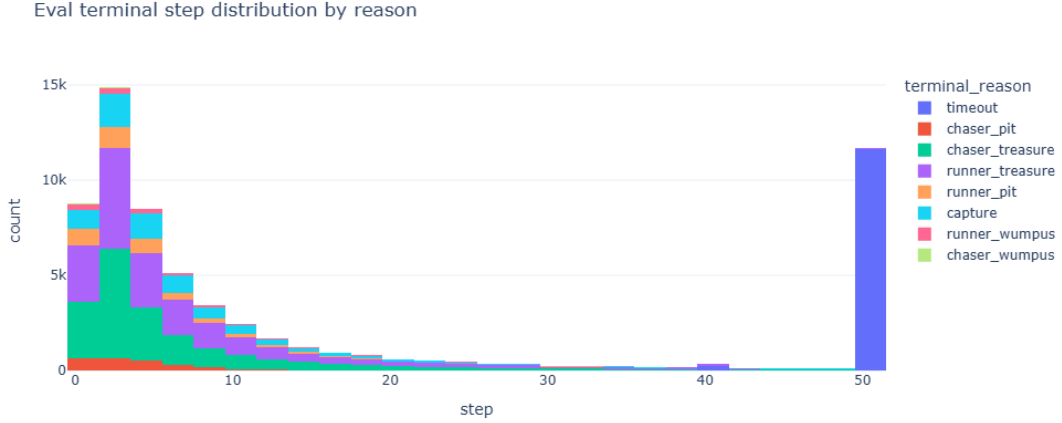


Figure 6.11: Terminal step distribution by terminal reason. Timeouts concentrate near the horizon, while capture/treasure endings occur substantially earlier.

Train vs evaluation behavioral gap

Figure 6.17 compares training and evaluation behavior via the Runner “opening-rate” gap, defined as $\Delta = \text{eval} - \text{train}$. The gap is consistently negative across runs, meaning that in evaluation the Runner increases distance less frequently than during training. The magnitude of this reduction varies by matchup, with the most pronounced drop observed in `dqn_vs_dqn`, while `fp_selfplay` is closer to zero. This pattern suggests that evaluation conditions systematically reduce separation dynamics for the Runner and that the deep-RL baseline is more sensitive to this shift.

6.2.4 Summary of findings

Across methods, Wumpus Chase performance is strongly role-dependent. `DynaQ` is the most effective Chaser, combining the highest Chaser win-rate with the only clearly positive Chaser terminal reward and relatively short episodes. `DynaQ+` is the strongest Runner, achieving the highest Runner win-rate and reward while frequently converting long-horizon survival into timeout wins. Terminal-reason analysis further shows that wins are not explained by a single mechanism: Chaser success is often associated with treasure endings, while Runner success is frequently associated with timeouts. Finally, hazard outcomes are pit-dominated in the tested layouts, emphasizing that pit management is the primary hazard-avoidance challenge in this scenario.

6.3 Evaluation Results: Opponent Diversity and Robustness

This section reports the supplementary Wumpus Chase experiment specified in Section 5.9. All conditions use a Dueling Double DQN learner and differ in their training opponents and action-failure probabilities, as defined in Table 5.1. The pilot uses the compact 5×5 map and a 30-step horizon. Its results are specific to this protocol and should be interpreted separately from the algorithm comparisons in Section 6.2.

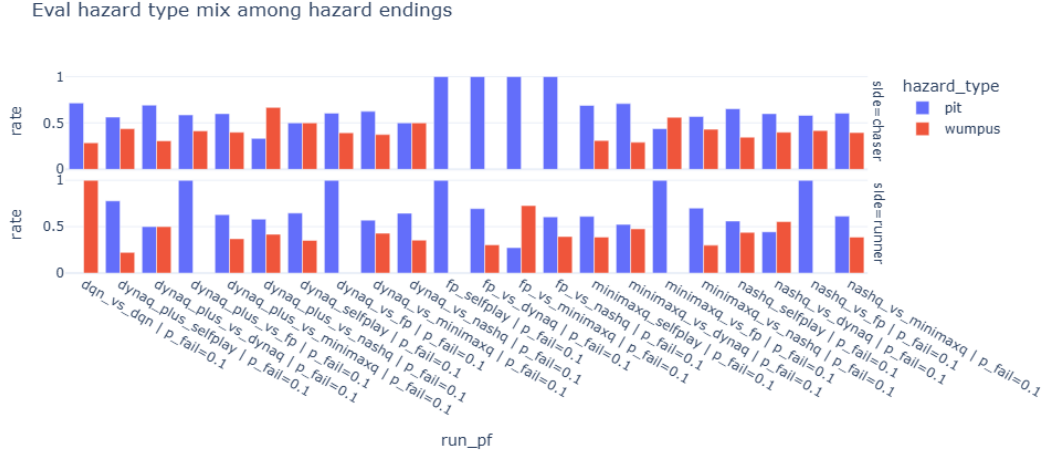


Figure 6.12: Hazard-type mix among hazard endings (pit vs Wumpus), split by side (Chaser vs Runner).

The primary endpoint averages win rates against the withheld treasure and ambush opponent families at $p_{\text{fail}} \in \{0.20, 0.30\}$. The reported means and nominal 95% confidence intervals summarize three training seeds. Paired comparisons with condition A use the within-seed average of its fixed-FP and fixed-Minimax controls. Section 5.9.4 provides the complete evaluation protocol; Section 5.9.5 discusses its scope and further experiments.

6.3.1 Pilot Results and Role-Dependent Interpretation

Table 6.1 reports the primary endpoint from the executed pilot notebook. These percentages concern unfamiliar scripted opponents under increased action noise, rather than the complete evaluation suite.

Condition	Chaser win rate (%)	Runner win rate (%)
A-FP	45.2	34.6
A-MM	50.5	44.0
B	54.7	43.2
C-FP	45.3	35.3
C-MM	56.0	49.4
D	56.8	44.2
E	54.6	44.2

Table 6.1: Mean primary-endpoint win rates over three training seeds. Bold values identify the largest observed mean within each role and do not establish superiority over every other condition.

Population training with randomized noise (D) achieves the highest Chaser mean, 56.8%, with a 95% interval of [52.4, 61.1]%. Relative to the within-seed average of the two A controls, its paired improvement is 8.9 percentage points, with interval $[-4.8, 22.6]$. The positive mean therefore remains uncertain across training seeds. For the Runner, D reaches 44.2%, with interval [40.7, 47.6]%, and improves over the averaged A controls by 4.9 per-

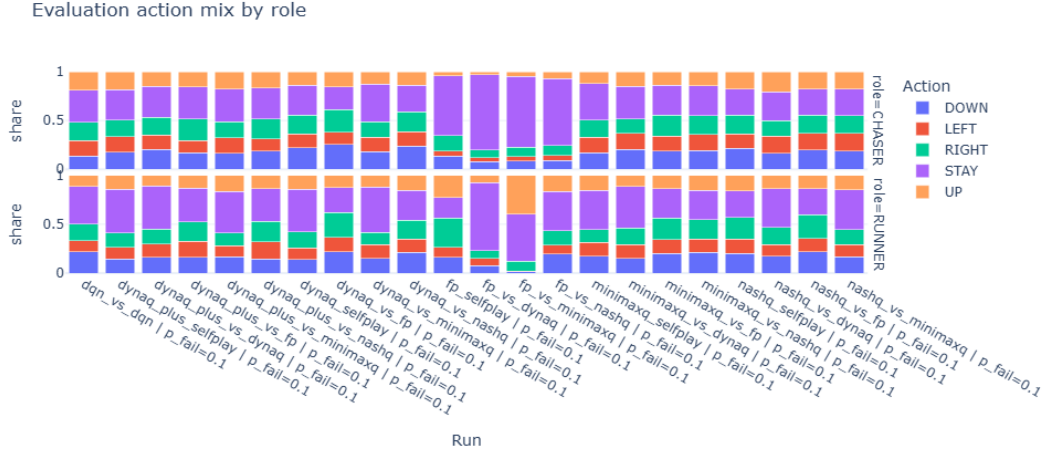


Figure 6.13: Evaluation action mix by role and run (stacked action shares).

centage points, with interval $[2.7, 7.0]$. This latter comparison supports an improvement relative to that specific averaged baseline under the pilot protocol.

The largest Runner mean is nevertheless obtained by C-MM at 49.4%. Thus, exposure to a diverse population does not dominate every carefully chosen fixed opponent. Adaptive sampling (E) also provides no clear additional benefit over D: the paired differences are -2.2 percentage points for the Chaser and $+0.1$ for the Runner, with intervals $[-9.9, 5.6]$ and $[-5.8, 6.0]$, respectively. Small differences between displayed means and contrasts arise from rounding.

The two role columns describe separately trained learners evaluated against role-specific opponent banks. They are not complementary outcomes from one shared set of matches and need not sum to 100%. Role asymmetry also follows from the task itself: capture benefits the Chaser, while survival to timeout benefits the Runner. Together with map geometry and the behavior represented in each opponent bank, these mechanisms can make the same training distribution more useful for one role than the other. The pilot does not isolate their individual causal contributions.

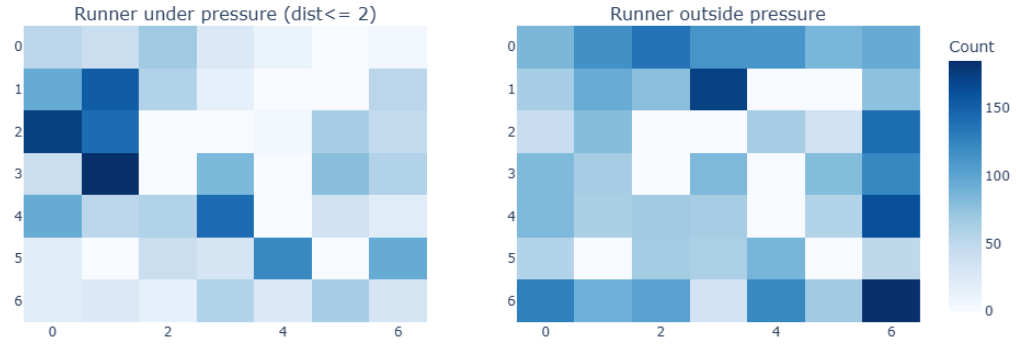
Control-zone heatmaps - dynaq_plus_selfplay, $p_{\text{fail}}=0.1$ 

Figure 6.14: Control-zone heatmaps for `dynaq_plus_selfplay`, $p_{\text{fail}} = 0.1$: Runner visitation density under pressure ($d_{AB} \leq 2$) vs outside pressure ($d_{AB} > 2$).

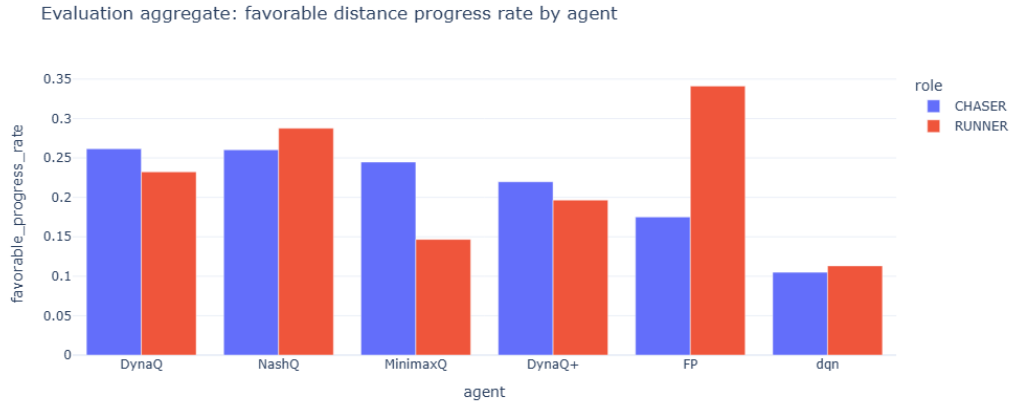


Figure 6.15: Evaluation aggregate: favorable distance progress rate by agent and role.

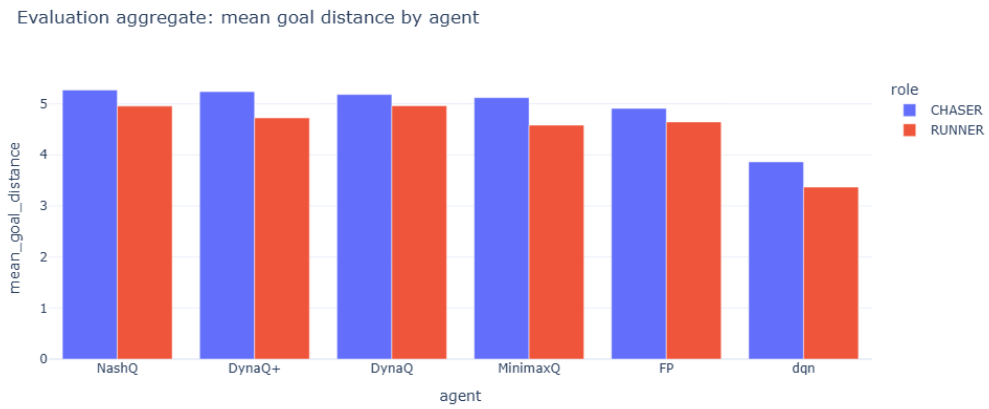


Figure 6.16: Evaluation aggregate: mean goal distance by agent and role.

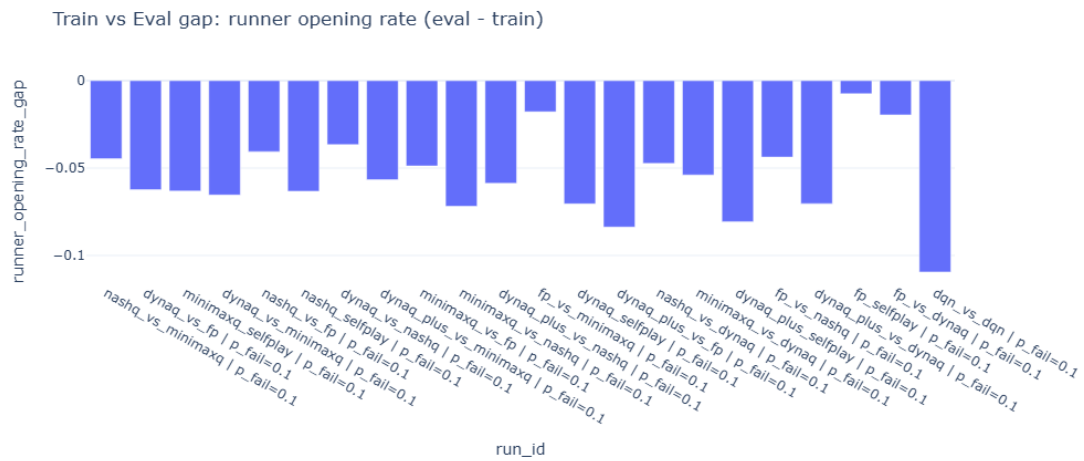


Figure 6.17: Train vs eval gap: Runner opening-rate difference (eval–train) by run id.

Chapter 7

Conclusion

This project investigated multi-agent reinforcement learning in adversarial grid-world pursuit–evasion, using two progressively more complex scenarios: a base Chase environment and an extended Wumpus Chase environment with hazards, perceptual cues, and an additional treasure objective. Across both settings, we implemented and evaluated a suite of model-free and game-theoretic methods (Fictitious Play, Minimax-Q, Nash-Q), and planning-based extensions (Dyna-Q and Dyna-Q+), complemented by a lightweight experimental DQN baseline.

In the base Chase scenario, evaluation results showed that learning dynamics are stable and largely robust to moderate actuation noise. Matchups with successful pursuit exhibited high capture rates and decreasing steps-to-capture over evaluation episodes, while specific pairings remained timeout-dominated. Spatial and distance-based diagnostics supported these findings by illustrating consistent pursuit/evasion patterns and comparable initial difficulty across runs.

In the Wumpus Chase scenario, the richer termination structure made outcome composition central to interpretation. Performance was strongly role-dependent: planning-based agents were consistently the strongest Chasers, while Dyna-Q+ emerged as the strongest Runner by frequently converting survival into timeout wins. Terminal-reason analysis showed that Chaser success often routes through treasure, whereas Runner success is frequently timeout-driven. Hazard analysis further indicated that pit-related terminations dominate over Wumpus-related ones in the tested layouts, emphasizing pit avoidance as the primary hazard-management challenge.

Overall, the experiments highlight that in adversarial environments with non-stationarity and multiple competing objectives, algorithmic choice interacts tightly with role semantics and termination logic. Planning-based updates (Dyna) provide a consistent advantage by improving sample efficiency and stabilizing behavior, while diagnostic decompositions (terminal reasons, action mix, distance progress, and spatial heatmaps) are essential for explaining *why* a method wins rather than only *how often* it wins.

Bibliography

- [1] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley Sons, 1994.
- [2] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition, 2018.
- [3] Lloyd S. Shapley. Stochastic games. *Proceedings of the National Academy of Sciences*, 39(10):1095–1100, 1953.
- [4] Michael L. Littman. Markov Games as a Framework for Multi-Agent Reinforcement Learning. In *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, pages 157–163. Morgan Kaufmann, 1994.
- [5] Junling Hu and Michael P. Wellman. Nash q-learning for general-sum stochastic games. *Journal of Machine Learning Research*, 4:1039–1069, 2003.
- [6] Lucian Busoniu, Robert Babuska, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, 38(2):156–172, 2008.
- [7] C. J. C. H. Watkins and Peter Dayan. Q-Learning. *Machine Learning*, 8(3-4):279–292, 1992.
- [8] Leslie P. Kaelbling, Michael L. Littman, and Andrew W. Moore. Reinforcement learning: A survey. *Journal of Artificial Intelligence Research*, 4:237–285, 1996.
- [9] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [10] Richard S. Sutton. Integrated architectures for learning, planning, and reacting based on approximating dynamic programming. In *Proceedings of the Seventh International Conference on Machine Learning*, pages 216–224, 1990.
- [11] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharmashan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.